

Algoritmi e strutture dati - Bonomi & D'Amore

Giovanni Esposito, Danilo Di Primio, Cristian Amendola

Università degli studi La Sapienza

Ingegneria Informatica e Automatica - 2 anno

Febbraio - Maggio 2026

Indice

1	Algoritmi	3
1.1	Costo di un algoritmo	3
	Complessità temporale	3
	Complessità spaziale	3
	Istruzione dominante	3
1.2	Andamento asintotico	3
	Analisi degli algoritmi	4
	Notazioni	4
1.3	Categorie di algoritmi	5
	Costante	5
	Cicli	6
	Ricorsione	6
1.4	Definizioni importanti	6
	Upper Bound	6
	Lower Bound	6
	Definizione di $\Omega(g(x))$	6
	O-grande di una funzione	7
	Definizione di $\theta(f(x))$	7
	Definizione di Istruzione Dominante	7
	Upper Bound problema	8
	Lower Bound problema	8
	Algoritmo ottimo	8
2	Algoritmi di ordinamento	9
2.1	Insertion Sort	9
	Pseudocodice InsertionSort	10
	Analisi dell'insertion sort	10
	Costi InsertionSort	11
2.2	Invariante di ciclo	12
2.3	Divide et impera	12
2.4	Merge sort	13
	Pseudocodice MergeSort	13
	Analisi del MergeSort	14
	Equazione di ricorrenza	14
2.5	Quick sort	15
	Pseudocodice QuickSort	16
	Dimostrazione della correttezza di Quicksort	17
	Prestazioni QuickSort	17
	Tempo di esecuzione QuickSort	17
2.6	Ricorrenze	17
2.7	Impostazione di una ricorrenza	18
2.8	Alberi di ricorsione per le ricorrenze	21
2.9	Alberi di decisione	23
	Teorema su alberi di decisione	24
	Teorema su algoritmi di ordinamento	24
2.10	Counting Sort	24
2.11	Bucket sort	25
	Pseudocodice BucketSort	25
	Analisi Bucket Sort	26

2.12	Radix Sort LSD	27
2.13	Riduzione (fra problemi)	27
2.14	HeapSort	28
	Definizione Heap	28
	Max-heap e Min-heap	28
	Max-Heapify	29
	Costruire un max-heap	30
	Heap Sort	31
3	Il problema della selezione e le statistiche d'ordine	32
3.1	Problema selezione	32
3.2	Minimo o Massimo	32
3.3	Minimo e Massimo	32
	Pseudocodice	33
3.4	Selezione Randomizzata	34
	Pseudocodice	35
	Analisi del costo	35
	Analisi del valore atteso del tempo di esecuzione	35
3.5	Selezione in tempo	38
	Pseudocodice	38
	Analisi	39
4	Programmazione Dinamica	40
4.1	Panoramica generale	40
4.2	Sviluppo dell'algoritmo	40
	Funzione di stato	40
	Top - Down	40
	Bottom Up	41
4.3	Problema dello zaino 0/1	41
4.4	Cammini su griglia (senza ostacoli)	43
5	Esercizi	44
5.1	Esercizi venerdì 13/03/2026	44

Capitolo 1

Algoritmi

Definizione 1.0.1. Un algoritmo è una procedura di calcolo ben definita che prende un certo valore (o un insieme di valori) come input e genera, in un tempo finito, un valore (o un insieme di valori) come output.

Un algoritmo è quindi una sequenza di passi computazionali che trasforma l'input nell'output.

1.1 Costo di un algoritmo

Definizione 1.1.1. Un algoritmo A ha costo $O(g(n))$ se la quantità di tempo (spazio) sufficiente per eseguire A su un input di dimensione n nel caso peggiore è $O(g(n))$.

Complessità temporale

La complessità temporale di un algoritmo è la misura del tempo di esecuzione di un algoritmo in funzione della dimensione dei dati in input. Questa complessità cresce al crescere della dimensione in input.

Complessità spaziale

La complessità spaziale di un algoritmo è una misura delle risorse hardware utilizzate durante l'esecuzione di un algoritmo. Se un algoritmo richiede più memoria per elaborare un insieme di dati più grande, allora ha una complessità spaziale più alta.

Istruzione dominante

Quando si analizza un algoritmo in cui sono presenti molti confronti, assegnazioni ecc..., è complicato calcolare il costo, in tal caso si prende in considerazione l'istruzione dominante, ovvero quell'operazione che si ripete più volte nel programma, tipicamente quella contenuta nei cicli più interni.

1.2 Andamento asintotico

L'andamento asintotico di un algoritmo è una misura della crescita del tempo di esecuzione o dello spazio richiesto quando le dimensioni dell'input crescono. In altre parole l'andamento asintotico ci dice come si comporta l'algoritmo quando l'input diventa grande, ovvero quando cresce la sua dimensione, il numero di bit di cui si ha bisogno di rappresentarlo (se l'input è pari ad n, la sua dimensione sarà pari a $\lceil \log_2(n+1) \rceil$). Il logaritmo in base 2 di un numero rappresenta esattamente il numero di bit necessari per rappresentarlo.

L'analisi ha le seguenti caratteristiche:

- **Asintotica** $\rightarrow$ non vogliamo sapere la funzione di costo esatta, ma vogliamo sapere il suo andamento asintotico ($f(x)$ per $x \rightarrow \infty$)

Due funzioni $f(x)$ e $g(x)$ hanno lo **stesso comportamento asintotico** quando $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = C > 0$

Algoritmo divisi in 2:

- Costo esponenziale
- Costo polinomiale

- Analisi asintotica di **caso peggiore** (worst case)
- Analisi asintotica della **dimensione in funzione dell'input**
 - Array in input con n celle, ogni cella ha k bit, dimensione input $k \cdot n$ essendo k costante mentre $n \rightarrow \infty$, k la definiamo costante moltiplicativa perciò non ci interessa.

L'algoritmo asintotico è utile perchè ci consente di fare previsioni sulle prestazioni di un algoritmo su input grandi senza dover eseguire l'algoritmo su input effettivamente grandi.

Il **comportamento asintotico** del costo di un programma si basa sull'idea di trascurare costanti moltiplicative e termini di ordine inferiore, fornendo un valore che cambia man mano che la dimensione aumenta. Ad esempio, il costo $a \cdot n + b$ ha comportamento asintotico lineare, il costo $a \cdot n^2 + b$ ha comportamento asintotico quadratico, il costo $a \cdot \log_b n + c$ ha comportamento asintotico logaritmico.

Per calcolare la complessità di un algoritmo si utilizza l'analisi asintotica che trasforma il tempo di esecuzione dell'algoritmo in una funzione $T(n)$ in funzione della dimensione n dei dati in input, con la quale misurare il numero di comandi eseguiti dall'algoritmo.

Analisi degli algoritmi

Analizzare un algoritmo significa prevedere le risorse che l'algoritmo richiederà, tra cui il tempo di elaborazione. Analizzando più algoritmi candidati a risolvere uno stesso problema, si può identificare quello più efficiente. Prima di analizzare un algoritmo, c'è bisogno di avere un modello della tecnologia di implementazione che sarà utilizzata e di un modo per esprimere i loro costi. Noi utilizzeremo il modello RAM (Random-Access Machine).

Notazioni

Ci sono delle notazioni principali utilizzate per esprimere l'andamento asintotico di un algoritmo:

Notazione O

La notazione O definisce un limite superiore al comportamento asintotico di una funzione. In altre parole, ci dice che una funzione non cresce più velocemente di un certo tasso di crescita, determinato dal termine di ordine superiore.

Esempio.

Data la funzione

$$7n^3 + 100n^2 - 20n + 6$$

il suo termine di ordine superiore è $7n^3$ e quindi diciamo che il tasso di crescita di questa funzione è n^3 , poichè questa funzione non cresce più velocemente di n^3 possiamo scrivere che è $O(n^3)$

Notazione Ω

La notazione Ω definisce un limite inferiore al comportamento asintotico di una funzione. In altre parole, ci dice che una funzione cresce almeno alla stessa velocità di un certo tasso di crescita.

Esempio.

Data la funzione

$$7n^3 + 100n^2 - 20n + 6$$

questa funzione è $\Omega(n^3)$, ma anche $\Omega(n^2)$ e $\Omega(n)$, più in generale è $\Omega(n^c)$ per ogni costante $c \leq 3$.

Notazione Θ

La notazione Θ definisce un limite stretto al comportamento asintotico di una funzione. Ci dice che una funzione cresce esattamente con un certo tasso di crescita.

Esempio.

Se riuscite a dimostrare che una funzione è al contempo $O(f(n))$ e $\Omega(f(n))$ per una qualche $f(n)$, allora avete dimostrato che la funzione è proprio $\Theta(f(n))$, nel nostro esempio è sia $\Omega(n^3)$ sia $O(n^3)$, allora è anche $\Theta(n^3)$.

Notazione o

Utilizziamo la notazione o per denotare un limite superiore che non è asintoticamente stretto.

Esempio.

$$2n = o(n^2), \text{ ma } 2n^2 \neq o(n^2)$$

Notazione ω

Utilizziamo la notazione ω per indicare un limite inferiore che non è asintoticamente stretto.

Esempio.

$$\frac{n^2}{2} = \omega(n), \text{ ma } \frac{n^2}{2} \neq \omega(n^2)$$

Proprietà

Proprietà transitiva

$$f(n) = \Theta(g(n)) \text{ e } g(n) = \Theta(h(n)) \text{ implicano } f(n) = \Theta(h(n))$$

$$f(n) = O(g(n)) \text{ e } g(n) = O(h(n)) \text{ implicano } f(n) = O(h(n))$$

$$f(n) = \Omega(g(n)) \text{ e } g(n) = \Omega(h(n)) \text{ implicano } f(n) = \Omega(h(n))$$

$$f(n) = o(g(n)) \text{ e } g(n) = o(h(n)) \text{ implicano } f(n) = o(h(n))$$

$$f(n) = \omega(g(n)) \text{ e } g(n) = \omega(h(n)) \text{ implicano } f(n) = \omega(h(n))$$

Proprietà riflessiva

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

Proprietà simmetrica

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Proprietà trasposta

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \iff g(n) = \omega(f(n))$$

1.3 Categorie di algoritmi

Dividiamo in 3 categorie le istruzioni degli algoritmi

- costante
- cicli
- ricorsioni

Costante

Al variare dell'input non cambia nulla.

Esempio.

$x = 10 \rightarrow$ non dipende dall'input, costo costante

Esempio.

$y = \frac{10+20}{5} \rightarrow$ costo non cambia al variare dell'input, costo costante

Esempio.

$z = x + 20y \rightarrow$ costo costante

Esempio.

return 0 $\rightarrow$ costo costante

Cicli

Attribuiamo il costo di un ciclo in base a quante volte viene eseguito il corpo del ciclo

Esempio.

```
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        ... costante  
        ... costante  
    }  
}
```

Il ciclo viene eseguito $n \cdot m$ volte, quindi il costo sarà proprio $n \cdot m$.

Esempio.

```
for (i=0; i<n; i++) {  
    for (j=0; j<i; j++) {  
        ... costante  
        ... costante  
    }  
}
```

In questo ciclo il costo è $\frac{(n-1)n}{2}$, quindi il costo sarà quadratico.

Esempio.

Nel caso in cui abbiamo 2 cicli:

- ciclo1 $\rightarrow$ costo: n^2
- ciclo2 $\rightarrow$ costo n

Il ciclo/cicli con costo più basso può essere trascurato

Ricorsione

R

```
----- -> costo costante  
----- -> costo costante  
R  
    ----- -> costo costante  
    ...
```

Il costo di una ricorsione dipende da quanti livelli ricorsivi (numero di chiamate ricorsive) vengono eseguiti.

1.4 Definizioni importanti

Upper Bound

Definizione 1.4.1. (Upper Bound) L'upper bound è un tetto al costo dell'algoritmo.

Quando un upper bound è il più basso possibile lo chiamiamo **tight**.

Lower Bound

Definizione 1.4.2. (Lower Bound) Il lower bound è un pavimento al costo dell'algoritmo.

Definizione di $\Omega(g(x))$

Definizione 1.4.3.

$$f(x) \in \Omega(g(x)) \Leftrightarrow \exists C > 0 : \forall n > n_0 \quad f(x) > C \cdot g(x)$$

O-grande di una funzione

Definizione 1.4.4. (O-grande)

$$f(n) \in O(g(n)) \Leftrightarrow \exists C > 0 : \forall n > n_0 \quad f(n) > C \cdot g(n)$$

Definizione di $\theta(f(x))$

Definizione 1.4.5.

$$f(x) \in \theta(g(x)) \Leftrightarrow f(x) \in O(g(x)) \wedge f(x) \in \Omega(g(x))$$

Definizione di Istruzione Dominante

Definizione 1.4.6. supponiamo che:

$$A \in O(f(n))$$

istruzione D con contributo espresso da $g(n)$:

$$\exists c > 0 : c \cdot g(n) > f(n)$$

Perchè facciamo l'analisi asintotica?

Con l'analisi asintotica non solo valutiamo l'andamento asintotico del costo ma possiamo trascurare tutte le costanti moltiplicative.

Esempio.

Funzione ordina (int primo, int ultimo);

```
int ordina ( int primo , int ultimo ) {  
    if( ultimo > primo ) infila ( primo , ordina ( primo + 1 , ultimo ) );  
    return ultimo ;  
}
```

Il costo della funzione è definito dalla seguente:

$$C_O = C_I + C_O$$

Il costo è quindi la somma della chiamata della funzione **infila** e la chiamata ricorsiva di **ordina**. Rappresentiamo il costo esplicitando i parametri:

$$C_O(a, b) = C_I(a, b) + C_O(a + 1, b) \quad \text{se } a < b$$

Se invece $a \leq b$ la funzione esegue il **return**, che è un'istruzione a costo costante C_1 .

Ora esplicitiamo la ricorsione:

$$C_O(0, n - 1) = C_I(a, b) + C_I(a + 1, b) + C_I(a + 2, b) + \dots + C_I(a + ..., b) + C_1$$

Che possiamo riscrivere con una sommatoria:

$$C_O(0, n - 1) = \sum_{i=0}^{n-2} C_I(i, n - 1) + C_1$$

Ora vediamo il costo della funzione **infila** (int i, int ultimo);

```
static void infila ( int i , int ultimo ) {  
    int h = i + 1;  
    int k = A [ i ];  
    while ( ( h <= ultimo ) && ( A[ h ] < k ) ) {  
        A [h -1] = A [ h ];  
        h ++;  
    }  
    A [h -1] = k ;  
}
```

L'istruzione dominante è un ciclo while, quindi ci concentriamo su quello.
 Il costo di un ciclo while è il massimo numero di iterazioni che può compiere

$$\text{Numero di iterazioni : } x + 1 \dots y = y + 1 - (x + 1) = y - x$$

Il costo di **infilare** sarà quindi il numero di iterazioni più una costante che rappresenta il costo delle istruzioni a costo costante fuori dal ciclo.

$$C_I(x, y) = y - x + C_2$$

Avendo calcolato il costo di C_I , possiamo sostituire questo risultato al costo di **ordina** che avevamo trovato:

$$C_O(0, n-1) = \sum_{i=0}^{n-2} (\underbrace{n-1}_{y} - \underbrace{i}_{x} + C_2) + C_1$$

Per risolvere questa sommatoria applico la proprietà distributiva e divido la sommatoria in due sommatorie, dove nella prima compaiono i termini costanti e nella seconda i termini che vengono incrementati.

$$\sum_{i=0}^{n-2} (n-1-i+C_2) + C_1 = \sum_{i=0}^{n-2} (n-1+C_2) - \sum_{i=0}^{n-2} i$$

Per la prima sommatoria visto che sono tutti termini costanti vuol dire che l'espressione dentro la sommatoria si ripeterà $n-1$ volte. Per la seconda usiamo il criterio di Gauss per la somma dei numeri naturali.

$$\sum_{i=0}^{n-2} (n-1+C_2) - \sum_{i=0}^{n-2} i = (n-1)(n-1+C_2) - \frac{(n-2)(n-1)}{2}$$

Per calcolare il costo basta fare questo calcolo:

$$(n-1)(n-1+C_2) - \frac{(n-2)(n-1)}{2} = \frac{n^2 - 2n(2-C_2) + 2(1-C_2) - n^2 + 3n - 2}{2}$$

L'algoritmo ha quindi complessità ordine n^2 .

Upper Bound problema

Definizione 1.4.7. Un problema P ammette

$$\begin{aligned} &UB \ O(f(n)) \\ &\Leftrightarrow \\ &\exists A(\text{che risolve P}) : UB(A) \in O(f(n)) \end{aligned}$$

Lower Bound problema

Definizione 1.4.8. Un problema P ammette

$$\begin{aligned} &\Omega \ g(f(n)) \\ &\Leftrightarrow \\ &\forall A(\text{che risolve P}) : LB(A) \in \Omega(g(n)) \end{aligned}$$

Algoritmo ottimo

Definizione 1.4.9. Quando LB e UB coincidono, l'algoritmo è definito OTTIMO.

Capitolo 2

Algoritmi di ordinamento

Il problema dell'ordinamento è un problema molto comune e spesso è il passo base per risolvere problemi più complessi.

Esempio.

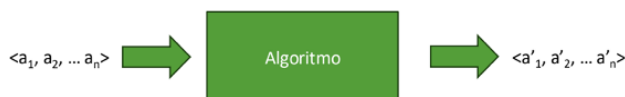
Vogliamo mettere in ordine alfabetico una lista di studenti prenotati ad un esame, perciò è necessario avere:

- un insieme di elementi
- un criterio in base a cui vogliamo ordinare

Definiamo il problema dell'ordinamento come segue:

input → una sequenza di n elementi $\langle a, b, c, \dots, d \rangle$

output → una permutazione $\langle a', b', c', \dots, d' \rangle$ della sequenza di input tale che $a' \leq b' \leq \dots \leq d'$



Definizione 2.0.1. (Istanza) L'istanza di un problema è formata dall'input richiesto per calcolare una soluzione del problema.

Definizione 2.0.2. (Algoritmo corretto) Algoritmo si dice corretto relativamente ad un problema computazionale se, per ogni istanza fornita in input esso:

1. Termina
2. Restituisce una soluzione corretta per l'istanza considerata

Algoritmi In-Place

Alcuni algoritmi di ordinamento possono ordinare una lista senza crearne una nuova. Questi sono conosciuti come algoritmi di ordinamento in loco (o in place) e richiedono uno spazio di memoria extra costante $O(1)$. InsertionSort e QuickSort sono IN-PLACE, in quanto gli elementi vengono spostati internamente all'array e non allocano memoria aggiuntiva. MergeSort non è IN PLACE, richiede memoria aggiuntiva per i diversi frammenti generati.

Definizione 2.0.3. (Definizione 1) Un algoritmo si definisce IN PLACE quando è in grado di restituire l'output utilizzando soltanto un piccolo e costante spazio di memoria extra, i dati in ingresso sono solitamente sovrascritti.

Definizione 2.0.4. (Definizione 2) Un algoritmo si dice IN PLACE se al crescere dell'input non crescono le variabili di appoggio.

2.1 Insertion Sort

Insertion Sort è un algoritmo di ordinamento molto semplice che ordina l'insieme di input come farebbe un umano per un mazzo di carte

1. inizialmente tutte le carte sono nel mazzo disordinato (input).
2. prendiamo la prima carta e la teniamo con la mano sinistra.
3. prendiamo la seconda carta, la confrontiamo con la prima e la posizioniamo alla sua destra o alla sua sinistra in base alla loro relazione d'ordine.
4. prendiamo la terza carta, la confrontiamo con quelle che abbiamo in mano e la sistemiamo nel posto giusto.
5. ripetiamo l'operazione di prendere una nuova carta e sistemarla nel posto giusto finché il mazzo non è finito.

Pseudocodice InsertionSort

:

```

INSERTION-SORT(A,n)
for i = 2 to n
    key = A[i]
    // inserisce A[i] nella sequenza ordinata A[1:i-1]
    j = i - 1
    while j > 0 and A[j] > key
        A[j + 1] = A[j]
        j = j - 1
    A[j + 1] = key

```

L'indice i identifica la carta corrente che viene inserita nella mano sinistra. All'inizio di ogni iterazione del ciclo `for`, il cui indice è i , il sotto-array formato dagli elementi $A[1 : i - 1]$ costituisce la mano di carte correntemente ordinate, mentre il sotto-array $A[i + 1 : n]$ corrisponde alla pila di carte ancora sul tavolo. In effetti, gli elementi $A[1 : i - 1]$ sono quelli che originariamente occupavano le posizioni da 1 a $i - 1$, ma che adesso sono ordinati. Definiamo formalmente queste proprietà soddisfatte da $A[i : i - 1]$ in forma di invariante di ciclo.

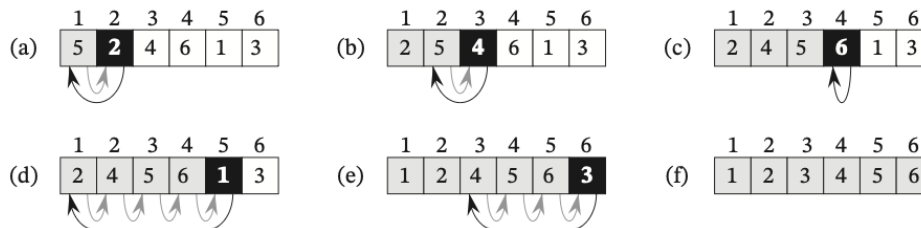


Figura 2.2 Il funzionamento della procedura $\text{INSERTION-SORT}(A, n)$ in cui originariamente A contiene la sequenza $\langle 5, 2, 4, 6, 1, 3 \rangle$ e $n = 6$. Gli indici dell'array sono indicati sopra i rettangoli; i valori memorizzati nelle posizioni dell'array sono indicati all'interno dei rettangoli. (a)-(e) Le iterazioni del ciclo `for` (righe 1-8). In ogni iterazione, il rettangolo nero contiene la chiave estratta da $A[i]$, che viene confrontata con i valori nei rettangoli grigi alla sua sinistra nel test della riga 5. Le frecce grigie mostrano i valori dell'array che vengono spostati di una posizione verso destra nella riga 6; le frecce nere indicano dove viene spostata la chiave nella riga 8. (f) L'array finale ordinato.

Analisi dell'insertion sort

Per prima cosa, dobbiamo comprendere che il tempo di esecuzione dipende dall'input, ovviamente ordinare più numeri richiederà più tempo. Per analizzare la procedura `insertion-sort`, la scriviamo qua sotto indicando a fianco di ogni istruzione il costo temporale e il numero di volte per cui viene eseguita.

$\text{INSERTION-SORT}(A, n)$	costo	numero di iterazioni
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 // Inserisce $A[i]$ nel sottoarray // ordinato $A[1 : i - 1]$.	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

Il tempo di esecuzione dell'algoritmo è la somma dei tempi di esecuzione per ogni istruzione eseguita; un'istruzione che richiede c_k passi viene eseguita m volte contribuirà con $c_k \cdot m$ al tempo di esecuzione totale. Indicheremo il tempo di esecuzione di un algoritmo su un input di dimensione n con $T(n)$. Per calcolare $T(n)$, il tempo di esecuzione di insertion-sort con un input di n valori, sommiamo i prodotti delle colonne costo e numero di volte ottenendo:

$$T(n) = c_1n + c_2(n-1) + C_4(n-1) + c_5 \sum_{i=2}^n (t_i - 1) + c_8(n-1)$$

Per insertion-sort il caso migliore si verifica quando l'array è già ordinato. Il tempo di esecuzione nel caso migliore è

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) = (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$

Questo tempo di esecuzione può essere espresso come $an+b$, dove a e b sono costanti che dipendono dai costi c_k delle istruzioni; quindi la funzione che esprime il tempo di esecuzione è una funzione lineare rispetto a n . Il caso peggiore si verifica quando l'array è ordinato in senso inverso, cioè in ordine decrescente. il tempo di esecuzione della procedura insertion-sort nel caso peggiore è

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) = \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + (c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8)n - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

Questo tempo di esecuzione può essere espresso come $an^2 + bn + c$, dove a , b e c sono costanti che dipendono dai costi delle istruzioni c_k ; quindi è una funzione quadratica rispetto a n .

Noi ci concentreremo solo sulla ricerca del tempo di esecuzione nel caso peggiore per tre ragioni:

1. il tempo di esecuzione nel caso peggiore di un algoritmo fornisce un limite superiore al tempo di esecuzione su un qualsiasi input.
2. per alcuni algoritmi, il caso peggiore si verifica molto spesso.
3. il "caso medio" spesso ha un costo simile a quello peggiore.

La velocità con cui cresce il tempo di esecuzione, ossia il suo tasso di crescita, è quello che effettivamente ci interessa. Di conseguenza consideriamo soltanto il termine principale di una formula, in quanto i termini di ordine inferiore sono relativamente poco significativi per grandi valori n . Ignoriamo anche il coefficiente costante del termine principale, in quanto i fattori costanti sono meno significativi del tasso di crescita nel determinare l'efficienza computazionale per grandi input. per il tempo di esecuzione del caso peggiore dell'insertion-sort, se ignoriamo i termini di ordine inferiore e il coefficiente del termine principale, resta solo il fattore n^2 che fa parte del termine principale.

Utilizzeremo la notazione $\Theta(\text{costo})$ per evidenziare l'ordine di crescita del tempo di esecuzione. L'insertion sort ha un tempo di esecuzione nel caso peggiore pari a $\Theta(n^2)$.

Un algoritmo è considerato più efficiente di un altro se il suo tempo di esecuzione nel caso peggiore ha un tasso di crescita minore rispetto all'altro.

Costi InsertionSort

Analizzando lo pseudocodice, si può notare che i cicli risultano essere le operazioni più frequenti. Dunque il ciclo `for` viene eseguito $n-1$ volte, mentre il ciclo `while`, j volte, tuttavia osservando che il valore j viene inizializzato fuori dal `while` a $i-1$, si capisce che l'ultimo valore sarà proprio $j = (n-1) - 1 = n-2$. Da ciò è possibile intuire che il costo dell'algoritmo sarà $(n-1) \cdot (n-2)$, ovvero il costo asintotico sarà pari a $O(n^2)$.

Dobbiamo fornire una **prova di correttezza**, ossia una dimostrazione formale che non considera un input specifico ma ragiona sull'algoritmo, sulle proprietà dell'input e su come viene prodotto l'output

- esistono molte tecniche per dimostrare la correttezza di un algoritmo rispetto ad un problema, come la tecnica degli invarianti di ciclo

2.2 Invariante di ciclo

È una tecnica per dimostrare che un particolare algoritmo risolve correttamente un dato problema. Ci sono tre cose che devono essere dimostrate:

1. Inizializzazione $\rightarrow$ esiste una proprietà che vale prima dell'iterazione del ciclo
2. Mantenimento $\rightarrow$ se la proprietà vale prima del ciclo, allora rimane vera anche prima dell'iterazione successiva
3. Conclusione $\rightarrow$ il ciclo termina e la proprietà che preserviamo dimostra che il risultato è corretto

Definizione 2.2.1. (Invarianza di ciclo) Una dimostrazione di invarianza di ciclo è una forma di dimostrazione per induzione matematica, dove per provare che una proprietà è valida, si prova un caso base e un passo induttivo.

Nel nostro caso, dimostrare che l'invariante è vera prima della prima iterazione corrisponde al caso base, mentre dimostrare che l'invariante resta vera da un'iterazione all'altra corrisponde al passo induttivo.

Inizializzazione

Iniziamo dimostrando che l'invariante di ciclo è vera prima della prima iterazione del ciclo, cioè quando $i = 2$. Il sottoarray $A[i : i - 1]$ è formato dal solo elemento $A[1]$, che infatti è l'elemento originale in $A[1]$. Inoltre, questo sottoarray è ordinato, il che dimostra che l'invariante di ciclo vale prima della prima iterazione del ciclo.

Mantenimento

Passiamo alla seconda proprietà: dimostrare che ogni iterazione conserva l'invariante di ciclo. Informalmente, il corpo del ciclo for esterno opera spostando $A[i - 1]$, $A[i - 2]$, $A[i - 3]$ e così via di una posizione verso destra, finché non troverà la posizione appropriata per $A[i]$, in cui inserirà il valore di $A[i]$. Il sottoarray $A[1 : i]$ quindi è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 : i]$. Dunque, l'incremento di i per la successiva iterazione del ciclo for preserva l'invariante di ciclo.

Conclusione

Infine, esaminiamo che cosa accade quando il ciclo termina. La variabile del ciclo i viene inizializzata a 2 e aumenta di 1 a ogni iterazione. Non appena il valore di i eccede n nella linea 1, il ciclo termina. Detto in altro modo: il ciclo termina non appena i assume il valore $n + 1$. Sostituendo $n + 1$ a i nella formulazione dell'invariante del ciclo, si trova che il sotto-array $A[1 : n]$ consiste degli elementi originariamente presenti in $A[1 : n]$, ma ordinati. Pertanto l'algoritmo è corretto.

2.3 Divide et impera

Dato un problema possono esistere diversi algoritmi in grado di risolverlo. Generalmente gli algoritmi ricorsivi adottano un approccio del tipo divide et impera.

- Divide
 1. il problema viene diviso in un certo numero di sottoproblemi
 2. tali sottoproblemi sono istanze più piccole del problema originale
- Impera
 1. i sottoproblemi vengono risolti in modo ricorsivo
 2. si continua a dividere fino ad arrivare ad una istanza sufficientemente piccola da essere risolta direttamente
- Combina
 1. Le soluzioni dei sottoproblemi vengono combinate per formare la soluzione del problema originale

2.4 Merge sort

L'algoritmo merge sort segue da vicino la strategia divide et impera. A ogni passo, ordina un sotto-array $A[r : p]$, partendo dall'intero array $A[1 : n]$ e operando ricorsivamente su sotto-array sempre più piccoli.

- Divide
 1. Se la lista contiene più di un elemento spezzala in due sotto-liste grandi la metà. Per fare ciò calcola il centro q del sottoarray $A[p : r]$ e divide $A[p : r]$ nei sottoarray $A[p : q]$ e $A[q+1 : r]$.
- Impera
 1. Continua a dividere finché la lista non ha dimensione 1. Quindi ordina ciascuno dei sottoarray in modo ricorsivo utilizzando l'algoritmo merge sort
- Combina
 1. Fonde due sotto-liste ordinandole

Pseudocodice MergeSort

```
MERGE-SORT(A,p,r)
if p >= r                //zero o un solo elemento?
    return
q=(p+r)/2                //punto medio di A[p : r]
MERGE-SORT(A,p,q)         //ordina ricorsivamente A[p : q]
MERGE-SORT(A,q+1,r)       //ordina ricorsivamente A[q+1 : r]
//fonde A[p : q] con A[q+1 : r] in A[p : r]
MERGE(A,p,q,r)
```

pseudocodice merge (metodo, combina, che fonde due sotto-liste ordinate):

```
MERGE(A,p,q,r)
n_l = q - p + 1           //lunghezza di A[p : q]
n_r = r - q               //lunghezza di A[q+1 : r]
crea due nuovi array L[0 : n_l -1] e R[0 : n_r -1]
for i =0 to n_l -1        //copia A[p : q] in L[0 : n_l -1]
    L[i] = A[p + i]
for j = 0 to n_r -1       //copia di A[q+1 : r] in R[0 : n_r - 1]
    R[j] = A[q + j + 1]
i = 0                     //i è l'indice del più piccolo elemento che resta in L
j = 1                     //i è l'indice del più piccolo elemento che resta in R
k = p                     //k è la posizione di A in cui fare il prossimo inserimento
//finchè L e R contengono un elemento non ancora fuso,
//copia il più piccolo elemento non fuso in A[p : r]
while i < n_l
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else A[k] = R[j]
        j = j + 1
    k = k + 1
//avendo esaurito completamente uno tra L e R,
//copia il resto dell'altro nella porzione finale di A[p : r]
while i < n_l
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < n_r
    A[k] = R[j]
    j = j + 1
    k = k + 1
```

esempio:

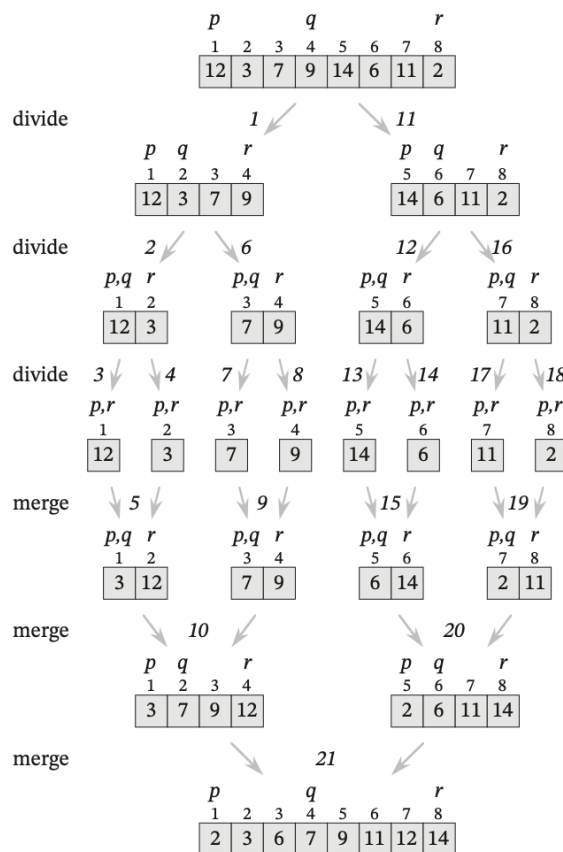


Figura 2.4

Funzionamento del merge sort su un array A di lunghezza 8 che inizialmente contiene la sequenza $\langle 12, 3, 7, 9, 14, 6, 11, 2 \rangle$. Gli indici p , q e r di ciascun sottoarray compaiono sopra i valori corrispondenti. I numeri in corsivo indicano l'ordine in cui vengono chiamate le procedure MERGE-SORT e MERGE dopo la chiamata iniziale $\text{MERGE-SORT}(A, 1, 8)$.

Analisi del MergeSort

Il MergeSort è un algoritmo efficiente la cui complessità temporale per l'ordinamento di un intero array di dimensione n è solamente $O(n \log(n))$.

L'algoritmo è conforme al paradigma Divide et Impera che viene sviluppato come segue:

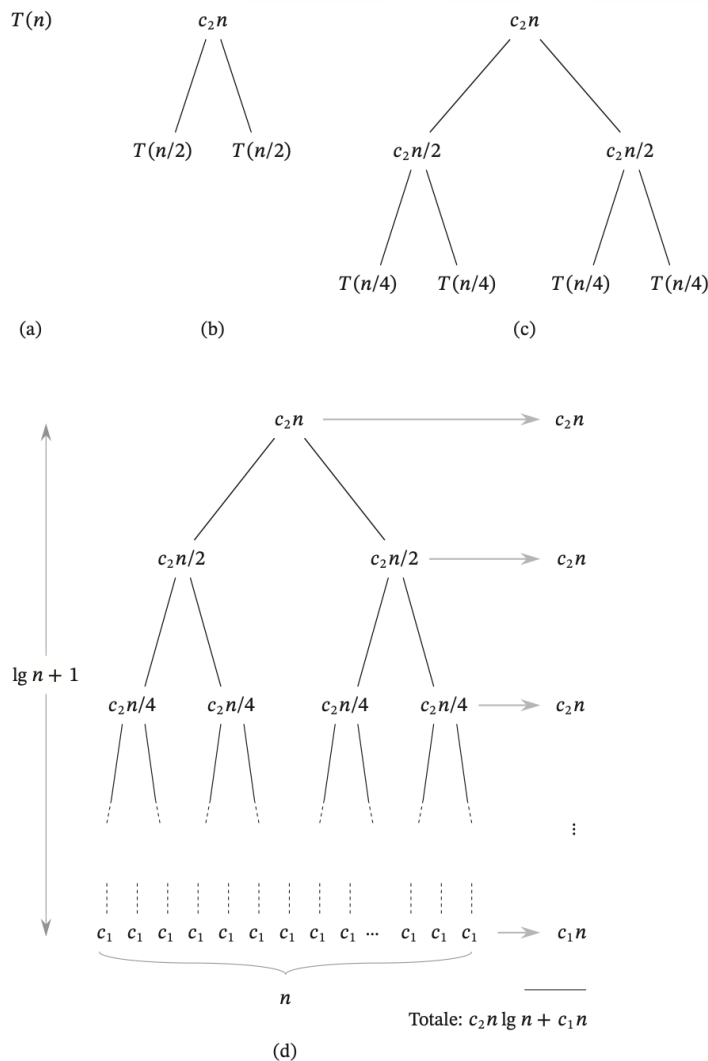
1. **Divide:** divide la sequenza degli n elementi da ordinare in due sotto-sequenze di $n/2$ elementi ciascuna.
2. **Impera:** ordina le due sotto-sequenze in modo ricorsivo.
3. **Combina:** fonde le due sotto-sequenze ordinate per generare la sequenza ordinata finale.

Equazione di ricorrenza

$$\begin{cases} T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n) & \text{passo ricorsivo} \\ T(1) = 1 & \text{passo base} \end{cases}$$

Figura 2.5

La costruzione di un albero di ricorsione per la ricorrenza $T(n) = 2T(n/2) + cn$. La parte (a) mostra $T(n)$, che viene progressivamente espanso in (b)-(d) per formare l'albero di ricorsione. L'albero completamente espanso nella parte (d) ha $\lg n + 1$ livelli. Ogni livello al di sopra di quello delle foglie contribuisce per un costo complessivo di c_2n , mentre il livello delle foglie ha un contributo di c_1n . Di conseguenza, il costo totale è $c_2n \lg n + c_1n = \Theta(n \lg n)$.



2.5 Quick sort

Il quicksort è un algoritmo di ordinamento il cui tempo di esecuzione nel caso peggiore è $\Theta(n^2)$ su un array di input di n numeri.

Questi sono i tre passi del processo divide et impera per ordinare un generico sottoarray $A[p : r]$:

- **Divide**

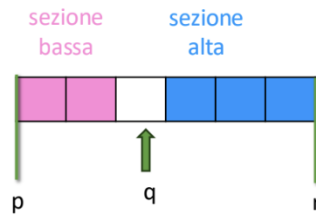
1. Fissiamo un pivot q
2. Riorganizziamo l'input in due sotto liste, $A[p : q - 1]$ (sezione bassa) e $A[q + 1 : r]$ (sezione alta)
3. Nella riorganizzazione dobbiamo garantire che ogni elemento della sezione bassa è $\leq$ di ogni elemento della sezione alta

- **Impera**

1. Ordiniamo le due sotto-liste in modo ricorsivo

- **Combina**

1. Poiché le due sotto liste sono già ordinate. Tutti gli elementi di $A[p : q - 1]$ sono in ordine e $\leq$ al pivot $A[q]$, e tutti quelli di $A[q + 1 : r]$ sono in ordine e $\geq$ al pivot $A[q]$. Ma allora l'intera lista $A[p : r]$ non può che essere ordinata.



Pseudocodice QuickSort

```

QUICKSORT(A,p,r)
1  if p < r
2      //Partiziona il sottoarray intorno al pivot, che finisce in A[q]
3      q = PARTIZIONA(A,p,r) //calcolo l'indice dell'elemento pivot con partiziona(A, p, r)
4      QUICKSORT(A,p,q-1) //ordina ricorsivamente la sezione bassa
5      QUICKSORT(A,q+1,r) //ordina ricorsivamente la sezione alta

```

Pseudocodice PARTIZIONA:

```

PARTIZIONA(A,p,r)
1  x = A[r] //il pivot
2  i = p - 1 //l'indice più alto nella sezione bassa
3  for j = p to r - 1 //tratta ogni elemento che non è il pivot
4      if A[j] <= x //questo elemento appartiene alla sezione bassa?
5          i = i + 1 //indice di nuova posizione della sezione bassa
6      scambia A[i] con A[j] //inserisce questo elemento nella nuova posizione
7  scambia A[i+1] con A[r] //il pivot si torva a destra della sezione bassa
8  return i + 1 //il nuovo indice del pivot

```

Esempio. Ordinare il seguente array:

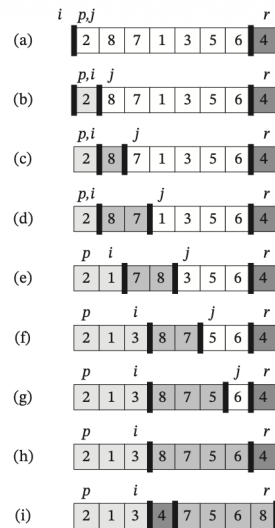


Figura 71 Il funzionamento di PARTITION su un array campione. L'elemento $A[r]$ dell'array diventa l'elemento pivot x . Gli elementi dell'array su sfondo grigio intermedio sono tutti nella sezione bassa e hanno valori non maggiori di x . Gli elementi su sfondo grigio scuro sono nella sezione alta e hanno valori maggiori di x . Gli elementi su sfondo bianco non sono stati ancora posti in una delle due sezioni; gli elementi su sfondo grigio scuro è il pivot x . **(a)** L'array iniziale e le impostazioni delle variabili. Nessuno degli elementi è stato posto in una delle due sezioni della partizione. **(b)** Il valore 2 viene "scambiato con se stesso" e posto nella sezione bassa. **(c)-(d)** I valori 8 e 7 vengono inseriti nella sezione alta. **(e)** I valori 1 e 8 vengono scambiati e la sezione bassa cresce. **(f)** I valori 3 e 7 vengono scambiati e la sezione bassa cresce. **(g)-(h)** La sezione alta della partizione acquisisce i valori 5 e 6 e il ciclo termina. **(i)** La riga 7 scambia il pivot in modo che cada tra le due sezioni della partizione e la riga 8 restituisce il nuovo indice del pivot.

All'inizio di ogni iterazione del ciclo for, per ogni indice k dell'array valgono le seguenti condizioni:

1. se $p \leq k \leq i$ allora $A[k] \leq x$
2. se $i + 1 \leq k \leq j - 1$ allora $A[k] \geq x$
3. se $k = r$ allora $A[k] = x$

Dimostrazione della correttezza di Quicksort

Dobbiamo dimostrare che questa invariante di ciclo è vera prima della prima iterazione, che ogni iterazione conserva l'invariante di ciclo, che il ciclo termina e, quando ciò avviene, che la correttezza segue dall'invariante.

Passo 1 - Inizializzazione

1. prima della prima iterazione del ciclo si ha $i = p - 1$ e $j = p$
2. poiché non ci sono valori fra p e i né fra $i + 1$ e $j - 1$, le prime due condizioni dell'invariante di ciclo sono banalmente soddisfatte.
3. l'assegnamento nella riga 1 soddisfa la terza condizione.

Passo 2 - Mantenimento

RECALL: Dobbiamo dimostrare che ogni iterazione mantiene l'invariante. Supponiamo quindi che sia vera prima della k -esima iterazione e mostriamo che resta vera al termine

Nella riga 11 di `partizione()` controllo se $A[k] \leq x$:

- CASO FALSE: non succede nulla e si passa alla prossima iterazione.
- CASO TRUE: viene incrementato l'indice i , vengono scambiati $A[i]$ e $A[j]$ e si passa alla prossima iterazione.

Passo 3 - Conclusione

1. il ciclo termina quando $j = r$
2. i valori dell'array si trovano ripartiti in tre insiemi: quelli minori o uguali a x , quelli maggiori di x e un insieme di un solo elemento che contiene x (il pivot).
3. viene eseguita la linea 6 del codice `PARTIZIONA(A, p, r)` e si scambiano $A[i + 1]$ e $A[r]$. L'output di `PARTIZIONA` adesso soddisfa le specifiche del passo divide. Infatti dopo la riga 3 di `QUICKSORT` $A[q]$ è strettamente minore di ogni elemento in $A[q + 1 : r]$.

Prestazioni QuickSort

il tempo di esecuzione del quicksort dipende da quanto è bilanciata ogni partizione e questo, a sua volta, dipende da quali elementi vengono usati come pivot. Se le due sezioni della partizione hanno all'incirca la stessa dimensione, cioè se è bilanciato, allora l'algoritmo ha un costo $O(n \log n)$. Se il partizionamento è sbilanciato ha un costo $O(n^2)$. Il costo in memoria dell'algoritmo invece è $O(n)$ in quanto non viene allocato nuovo spazio in memoria per i sotto-array, bensì si richiama ricorsivamente sullo stesso.

Il caso peggiore del QuickSort è quando l'array da ordinare è già ordinato o è ordinato in ordine inverso, o quando tutti gli elementi sono uguali.

Il QuickSort è di per sé un algoritmo efficiente, però può essere migliorato abbassando la probabilità di incontrare dei casi sfavorevoli, quindi evitando di incontrare casi di sbilanciamento nei due sotto-array. Per fare questo si utilizza una versione randomizzata del QuickSort, in cui la scelta del pivot avviene in modo pseudo-casuale.

Tempo di esecuzione QuickSort

$$T(n) = \begin{cases} O(1) & \text{se } n \leq 1 \text{ passo base} \\ T(k) + T(n - k - 1) + O(n) & \text{se } n > 1 \text{ passo ricorsivo} \end{cases}$$

Dove $T(k)$ rappresenta il tempo di esecuzione del QuickSort per il sotto array con k elementi, $T(n - k - 1)$ rappresenta il tempo di esecuzione del QuickSort per il sotto array con $n - k - 1$ che non includono il pivot scelto, $O(n)$ rappresenta il tempo necessario per effettuare la partizione dell'array in base al pivot, che richiede un tempo lineare proporzionale alla dimensione dell'array. Il passo base dell'equazione di ricorrenza del QuickSort è quando l'array da ordinare contiene zero o un solo elemento, in cui il tempo di esecuzione è costante.

2.6 Ricorrenze

Per analizzare gli algoritmi ricorsivi basati sul metodo divide et impera, abbiamo bisogno di alcuni strumenti matematici. Una **ricorrenza** è un'equazione che descrive una funzione in termini del suo valore su altri argomenti, in genere più piccoli. Le ricorrenze vengono usate con il metodo divide et impera perché ci danno un modo naturale per caratterizzare matematicamente i tempi di esecuzione degli algoritmi ricorsivi.

La forma generale di una ricorrenza è un'equazione o una disequazione che descrive una funzione definita sugli interi o sui reali utilizzando la funzione stessa. Essa contiene due o più casi, a seconda dell'argomento. Se un caso comporta la chiamata ricorsiva della funzione su input diversi, si tratta di un **caso ricorsivo**. Se un caso non comporta una chiamata ricorsiva, è un **caso base**.

Ricorrenze algoritmiche

Una ricorrenza $T(n)$ è di natura algoritmica se, per ogni soglia $n_0 > 0$ sufficientemente grande valgono le seguenti proprietà:

1. per ogni $n < n_0$, si ha $T(n) = \Theta(1)$
2. per ogni $n \geq n_0$, ogni cammino di ricorsione termina in un caso base entro un numero finito di chiamate ricorsive.

2.7 Impostazione di una ricorrenza

Tutte le volte che abbiamo un algoritmo ricorsivo, bisogna impostare una **ricorrenza**.

Esempio.

```
int ricercaBinariaRic(int a[], int primo, int ultimo, int k)
```

Vogliamo cercare una funzione $T(n)$ che descriva il costo della funzione, il nostro esempio ha questa ricorrenza:

$$\begin{cases} T(n) = C + T\left(\frac{n}{2}\right) & n > 1 \\ T(1) = 1 & \rightarrow \text{caso base} \end{cases}$$

Procediamo con uno sviluppo manuale per sostituzione, che chiamiamo per **Srotolamento**:

Dopo aver impostato la ricorrenza, applichiamo la manualmente la ricorsione:

$$n \rightarrow \frac{n}{2} : \quad T\left(\frac{n}{2}\right) = 1 + T\left(\frac{n}{2^2}\right)$$

$$\frac{n}{2} \rightarrow \frac{n}{4} : \quad T\left(\frac{n}{4}\right) = 1 + T\left(\frac{n}{2^3}\right)$$

Quindi riscriviamo l'espressione di $T(n)$ con le sostituzioni che abbiamo fatto sopra:

$$T(n) = 1 + \underbrace{1 + T\left(\frac{n}{2^2}\right)}_{T\left(\frac{n}{2}\right)} = 1 + 1 + \underbrace{1 + T\left(\frac{n}{2^3}\right)}_{T\left(\frac{n}{2^2}\right)} = \dots = \boxed{K + T\frac{n}{2^k}}$$

Visto che il caso base è $T(1) = 1$, dobbiamo vedere per quale valore di k si ha che $\frac{n}{2^k} = 1$. Ciò avviene per $k = \log n$, quindi riscriviamo l'espressione di $T(n)$ con questo valore di k :

$$T(n) = \log n + 1 \in \Theta(\log n)$$

Questo esempio è la dimostrazione di un algoritmo **In-Place**.

Negli algoritmi "in-place" si usa la dimensione dell'input più qualche variabile. Non tutti gli algoritmi possono essere portati "in-place" come ad esempio merge-sort.

Esempio.

```
MERGE-SORT(A,p,r)
if p >= r           //zero o un solo elemento?
    return
q=(p+r)/2           //punto medio di A[p : r]
MERGE-SORT(A,p,q)   //ordina ricorsivamente A[p : q]
MERGE-SORT(A,q+1,r) //ordina ricorsivamente A[q+1 : r]
//fonde A[p : q] con A[q+1 : r] in A[p : r]
MERGE(A,p,q,r)
```

Questo algoritmo ha questo schema di ricorrenza:

$$\begin{cases} T(n) = 2T\left(\frac{n}{2}\right) + M(n) \\ T(1) = 1 \rightarrow \text{caso base} \end{cases}$$

Dove $M(n)$ è il costo della chiamata del $\text{MERGE}(A, p, q, r)$, che ancora non conosciamo.

Per vedere quanto vale il costo guardiamo il suo pseudocodice:

```

MERGE(A, p, q, r)
nL = q - p + 1          //lunghezza di A[p : q]
nR = r - q              //lunghezza di A[q+1 : r]
//crea due nuovi array
L = [0 : nL - 1]
R = [0 : nR - 1]

for i = 0 to nL - 1      //copia A[p : q] in L[0 : nL - 1]
    L[i] = A[p + i]

for j = 0 to nR - 1      //copia di A[q+1 : r] in R[0 : nR - 1]
    R[j] = A[q + j + 1]
i = 0                    //i è l'indice del più piccolo elemento che resta in L
j = 0                    //j è l'indice del più piccolo elemento che resta in R
k = p                    //k è la posizione di A in cui fare il prossimo inserimento

//unisci i due sottoarray in ordine
while i < nL and j < nR
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else A[k] = R[j]
        j = j + 1
    k = k + 1

//avendo esaurito completamente uno tra L e R,
//copia il resto dell'altro nella porzione finale di A[p : r]
while i < nL
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < nR
    A[k] = R[j]
    j = j + 1
    k = k + 1

```

Dallo pseudocodice si evince che il costo di $M(n)$ è n , poichè quando vengono copiati gli array in due buffer temporanei, si ha un costo di $\Theta(n)$, poi quando vengono effettuati gli ultimi due cicli per svuotare i buffer, per ogni elemento "spendiamo" una costante, quindi anche in questa fase il costo è di $\Theta(n)$.

Possiamo quindi riscrivere lo schema di ricorrenza con $M(n) = n$:

$$\begin{cases} T(n) = 2T\left(\frac{n}{2}\right) + n \\ T(1) = 1 \end{cases}$$

Si procede con lo srotolamento:

$$\begin{aligned} n \rightarrow \frac{n}{2} : \quad T\left(\frac{n}{2}\right) &= 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \\ \frac{n}{2} \rightarrow \frac{n}{4} : \quad T\left(\frac{n}{2^2}\right) &= 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \end{aligned}$$

Quindi $T(n)$ diventa:

$$T(n) = 2T\left(\frac{n}{2}\right) + n = 2 \underbrace{\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right]}_{T\left(\frac{n}{2}\right)} + n = \dots = \boxed{2^k T\left(\frac{n}{2^k}\right) + k n}$$

Dato il caso base $T(1) = 1$, dobbiamo trovare il valore di k tale che $2^k = 1$, ovvero per $k = \log n$, perciò $T(n)$ diventa:

$$T(n) = 2^{\log n} T(1) + \log n \cdot n = n + n \log n \in \Theta(n \log n)$$

Esempio.

Si ha la seguente ricorrenza:

$$\begin{cases} T(n) = T(n-1) + n \\ T(1) = 1 \end{cases} \rightarrow \text{caso base}$$

Applichiamo lo srotolamento:

$$n \rightarrow n-1 : \quad T(n-1) = T(n-2) + n-1$$

$$n-1 \rightarrow n-2 : \quad T(n-2) = T(n-3) + n-2$$

Sostituendo a $T(n)$ diventa:

$$T(n) = T(n-3) + (n-2) + (n-1) + n = \dots = \boxed{T(n-k) + \sum_{i=0}^{k-1} (n-i)}$$

Semplifichiamo la sommatoria:

$$\sum_i^{k-1} (n-i) = \sum_i^{k-1} n - \sum_i^{k-1} i = k \cdot n - \frac{(k-1)(k-2)}{2}$$

dove per la seconda sommatoria abbiamo usato il criterio di Gauss.

Ora per il caso base $T(1) = 1$, troviamo che $n-k=1$ quando $k=n-1$, sostituendo si ottiene:

$$T(n) = (n-1) \cdot n - \frac{(n-2)(n-3)}{2} = 1 + \frac{2n^2 - 2n - n^2 - 3n - 2n - 6}{2} \in \Theta(n^2)$$

Esempio.

Si ha la seguente ricorrenza:

$$\begin{cases} T(n) = T\left(\frac{n}{2}\right) + \frac{n}{2} \\ T(1) = 1 \end{cases} \rightarrow \text{caso base}$$

Applichiamo lo srotolamento:

$$n \rightarrow \frac{n}{2} : \quad T\left(\frac{n}{2}\right) = T\left(\frac{n}{2^2}\right) + \frac{n}{2^2}$$

$$\frac{n}{2} \rightarrow \frac{n}{4} : \quad T\left(\frac{n}{2^2}\right) = T\left(\frac{n}{2^3}\right) + \frac{n}{2^3}$$

Sostituendo a $T(n)$ diventa:

$$T(n) = \underbrace{T\left(\frac{n}{2^2}\right) + \frac{n}{2^2} + \frac{n}{2}}_{T\left(\frac{n}{2}\right)} = \dots = \boxed{T\left(\frac{n}{2^k}\right) + n \sum_{i=1}^k \left(\frac{1}{2}\right)^i}$$

Per il caso base $\frac{n}{2^k} = 1$ per $k = \log n$, inoltre la sommatoria è una successione geometrica di ragione $q = \frac{1}{2}$, quindi la sua somma è nota. Sostituiamo e semplifichiamo:

$$T(n) = 1 + n \frac{q^{k+1} - 1}{q - 1} = 1 + 2n \left(1 - \frac{1}{2^{\log n}}\right) = 1 + 2n \left(1 - \frac{1}{n}\right)$$

2.8 Alberi di ricorsione per le ricorrenze

Oltre al metodo di sostituzione, un altro modo per analizzare il costo di un algoritmo è usare gli **alberi di ricorsione**. In questa struttura, ogni nodo rappresenta il costo di un singolo sotto-problema di una delle chiamate ricorsive della funzione. Questo metodo consiste nel sommare i costi dei nodi che si trovano sullo stesso livello per ottenere il costo per livello. Poi sommeremo il costo di ogni livello per determinare il costo totale dell'algoritmo ricorsivo.

Esempio.

Prendiamo in esame l'algoritmo Quicksort, che ha il seguente pseudo-codice:

```
1  QUICKSORT(A,p,r)
2  if p < r
3      q = PARTIZIONA(A,p,r)  //calcolo l'indice dell'elemento pivot con partiziona(A, p, r)
4      QUICKSORT(A,p,q-1)    //ordina ricorsivamente la sezione bassa
5      QUICKSORT(A,q+1,r)    //ordina ricorsivamente la sezione alta
6  end
7  PARTIZIONA(A,p,r)
8  x = A[r]                  //il pivot
9  i = p - 1                 //l'indice più alto nella sezione bassa
10 foreach j in [p : r-1] do //tratta ogni elemento che non è il pivot
11     if A[j] <= x then      //questo elemento appartiene alla sezione bassa?
12         i = i + 1          //indice di nuova posizione della sezione bassa
13         scambia A[i] con A[j] //inserisce questo elemento nella nuova posizione
14     end
15 end
16 scambia A[i+1] con A[r]    //il pivot si torva a destra della sezione bassa
17 return i + 1              //il nuovo indice del pivot
```

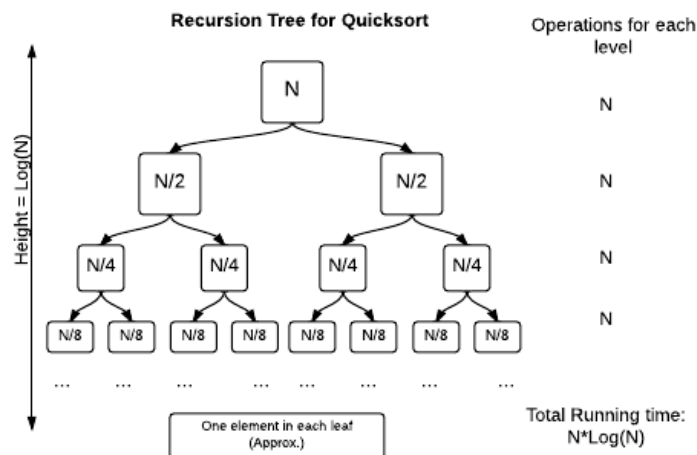
Impostiamo la ricorrenza, che terrà della chiamata ricorsiva e della chiamata alla funzione $\text{PARTIZIONA}(A, p, r)$:

$$T(n) = 2T(n-1) + P(n)$$

Vediamo prima il costo di $\text{PARTIZIONA}(A, p, r)$, dallo pseudo-codice si evince che sia:

$$P(n) \in \Theta(n) \rightarrow P(n) = c \cdot n$$

Vediamo prima cosa succede nel **caso migliore**, ovvero nel caso in cui scegliamo il pivot che divide perfettamente a metà l'array da ordinare. Avremo perciò il seguente albero di ricorsione.



Quindi si ha che:

- **Radice:** Nel primo nodo più alto c'è il problema iniziale: ordinare un array di n elementi
- **Partizione:** l'algoritmo sceglie un pivot e divide l'array in due sotto-array di $\frac{n}{2}$ elementi:
 - elementi minori del pivot
 - elementi maggiori del pivot
- **Livelli successivi:** ogni sotto-array viene ordinato con un'altra chiamata a Quicksort, quindi l'albero continua:
 - livello 0 $\rightarrow n$
 - livello 1 $\rightarrow \frac{n}{2}$
 - livello 2 $\rightarrow \frac{n}{4}$
 - livello 3 $\rightarrow \frac{n}{8}$
- **Foglie dell'albero:** L'algoritmo si ferma quando i sotto-array hanno 1 elemento

Una volta impostato l'albero di ricorsione basterà quindi sommare i costi di ogni livello e successivamente sommare il costo di ogni livello:

→ Livello 0: c'è un unico nodo di costo N

→ Livello 1: ci sono due nodi di costo $\frac{N}{2}$, quindi il costo del livello è $2 \cdot \left(\frac{N}{2}\right) = N$

→ Livello 2: ci sono quattro nodi di costo $\frac{N}{4}$, quindi il costo del livello è $4 \cdot \left(\frac{N}{4}\right) = N$

⋮

→ Livello k : ci sono k nodi di costo $\frac{N}{k}$, quindi il costo del livello è $k \cdot \left(\frac{N}{k}\right) = N$

Sommando i costi di ogni livello si ottiene che il costo dell'algoritmo nel caso migliore è:

$$T(n) = N \cdot k$$

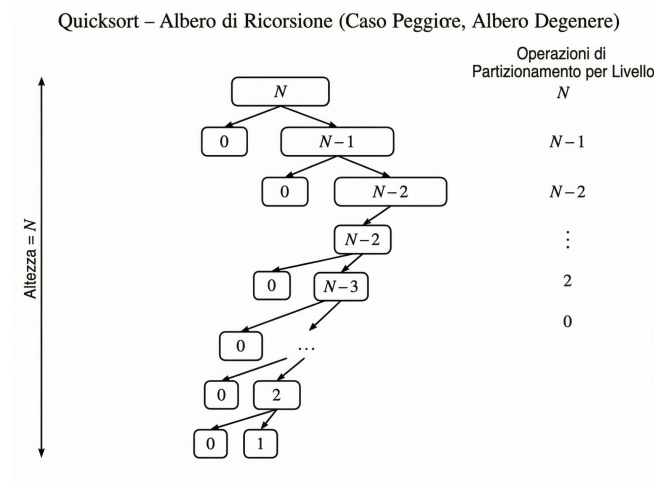
Ma quanto vale k ?

Per capire quanto vale k (ovvero quanto è alto l'albero della ricorsione) ci dobbiamo chiedere quante volte il valore N può essere diviso per due fino a raggiungere 1. Ciò equivale a dire a quale esponente si deve elevare 2 per ottenere N , e questa è proprio la definizione di logaritmo in base 2 di N .

Quindi il costo totale dell'algoritmo nel caso migliore è:

$$T(n) = N \cdot \log_2 N$$

Ora vediamo il **caso peggiore**: ciò succede quando o la lista è già ordinata, oppure quando viene scelto un pivot che sia il primo o l'ultimo elemento dell'array. Infatti l'albero di ricorsione sarà degenerare (ogni nodo ha al massimo un figlio), poichè l'array viene suddiviso in un sotto-array di dimensione $n - 1$, e un sotto-array di 1 elemento. L'albero di ricorsione perciò sarà così:



In questo caso il costo totale è evidente, in quanto l'albero di ricorsione è alto N e ad ogni livello il costo viene diminuito di 1:

$$T(n) = N \cdot \sum_{i=0}^k (N - i) \rightarrow \boxed{T(n) \in O(n^2)}$$

Il **caso medio** invece, tiene conto della probabilità con cui viene scelto il pivot. Infatti c'è sempre la probabilità del 50% di scegliere un pivot "buono" e il restante 50% di scegliere un pivot "cattivo". Nel caso medio non ci troviamo nè nel caso migliore nè nel caso peggiore, quindi l'altezza dell'albero di ricorsione sarà sempre $N \cdot \log N$, ciò che cambia rispetto al caso migliore è che la base del logaritmo non è più 2, ma dipenderà dalla scelta dei pivot. In generale, comunque, possiamo affermare che il costo nel caso medio è:

$$\boxed{T(n) = N \cdot \log N}$$

2.9 Alberi di decisione

Definizione 2.9.1. (Albero di decisione) Un albero di decisione è un albero pieno (cioè ogni nodo è una foglia oppure ha entrambi i figli), che rappresenta i confronti fra elementi che vengono effettuati da un algoritmo che agisce su un input di una data dimensione.

Osservazione. Gli alberi di decisione non sono strutture dati.

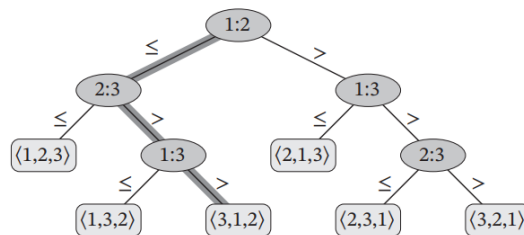
L'esecuzione dell'algoritmo di ordinamento corrisponde a tracciare un cammino semplice a partire dalla radice dell'albero di decisione fino ad arrivare ad una foglia. Ogni nodo rappresenta un confronto $a_i \leq a_j$. Il sotto-albero sinistro rappresenta i successivi confronti da fare se la condizione $a_i \leq a_j$ risulta vera, mentre il sotto-albero destro è dedicato al caso in cui $a_i \leq a_j$ non è verificata. Raggiunta una foglia l'algoritmo ha definito un certo ordinamento $a_1 \leq a_2 \leq \dots \leq a_n$.

Siccome qualsiasi algoritmo di ordinamento corretto deve essere in grado di produrre ogni permutazione del suo input, allora una **condizione necessaria** affinché un ordinamento per confronti sia corretto è che ciascuna delle $n!$ permutazioni di n elementi appaia almeno una volta in una delle foglie dell'albero di decisione. Inoltre ciascuna di queste foglie deve essere raggiungibile dalla radice, attraverso un percorso che corrisponde a una effettiva esecuzione dell'ordinamento per confronti.

Esempio.

Vogliamo ordinare un'array di 3 elementi tramite l'algoritmo di Insertion Sort.

L'albero di decisione per questo algoritmo è il seguente:



Osservazione. I numeri che sono nei nodi non sono i numeri di con cui sta avvenendo il confronto, ma sono gli indici dell'array di 3 elementi.

Una volta impostato l'albero di decisione verifichiamo la condizione necessaria, quindi controlliamo che nelle foglie ci siano tutte le possibili permutazioni dell'input: la dimensione dell'array è $n = 3$, quindi ci aspettiamo che nelle foglie ci siano tutte le $n! = 3! = 6$ combinazioni dell'array. Si nota che le 6 permutazioni sono presenti nelle foglie e che sono tutte raggiungibili a partire dalla radice, quindi la condizione necessaria è soddisfatta. Ovviamente il percorso dalla radice alla foglia più corto rappresenta il caso migliore dell'algoritmo, analogamente il percorso più lungo (che corrisponde all'altezza dell'albero) rappresenta il caso peggiore.

Teorema su alberi di decisione

Teorema 2.9.1. Dato un albero di decisione, sia

- N : il numero di nodi
- F : il numero di foglie

allora

$$N = 2F - 1$$

Esempio.

Prendiamo in considerazione l'esempio di prima, l'albero di decisione ha 6 foglie, quindi applicando la formula del teorema il numero di nodi è:

$$N = 2F - 1 = 2 \cdot 6 - 1 = \boxed{11}$$

Teorema su algoritmi di ordinamento

Teorema 2.9.2. Ogni algoritmo di ordinamento per confronti richiede $\Omega(n \log n)$ confronti nel caso peggiore.

Dimostrazione. Per stabilire il numero di confronti nel caso peggiore, è sufficiente sapere quanto è alto l'albero di decisione. Consideriamo un albero di decisione di un algoritmo che ha in input n elementi e poniamo:

- H : valore dell'altezza dell'albero
- L : numero di foglie

Poichè ciascuna delle $n!$ permutazione deve comparire almeno in una foglia (condizione necessaria), si ha che

$$n! \leq L$$

Inoltre sappiamo che un albero binario di altezza H non ha più di 2^H foglie, si ha che

$$n! \leq L \leq 2^H$$

Considero la disuguaglianza $n! \leq 2^H$ e facendo il logaritmo in base 2 da tutte e due le parti si ottiene:

$$H \geq \lg(n!) \xrightarrow{\text{Approssimazione di Stirling}} \boxed{H = \Omega(n \log n)}$$

□

Sorting non basato sul confronto

2.10 Counting Sort

L'algoritmo Counting Sort suppone che ciascuno degli n elementi di input sia un numero intero compreso tra 0 e k , per qualche intero k . Il Counting Sort per prima cosa determina, per ogni elemento di input x , il numero di elementi minori o uguali a x , e usa questa informazione per inserire l'elemento x direttamente nella sua posizione nell'array di output. Ecco lo pseudo-codice del Counting-Sort:

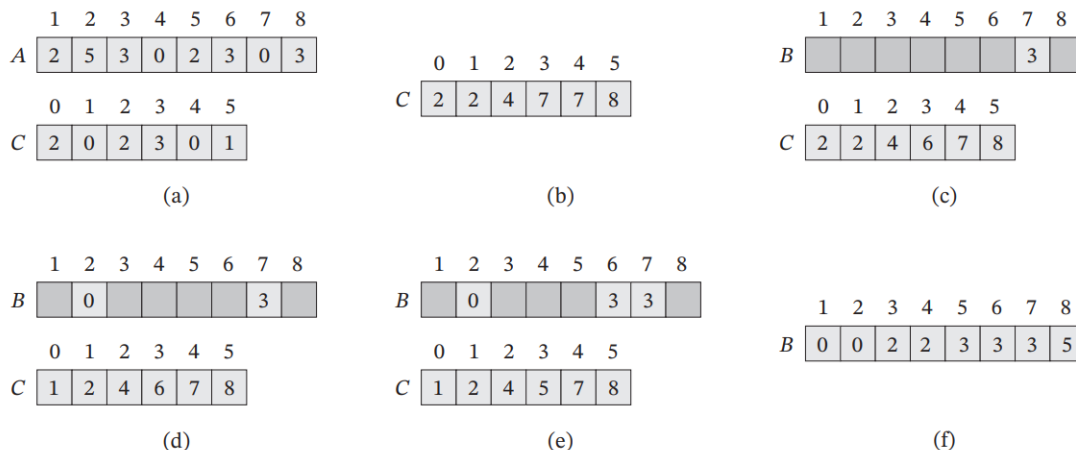
COUNTING-SORT(A, n, k)

```
1  siano B[1 : n] e C[0 : k] due nuovi array
2  for i = 0 to k
3      C[i] = 0
4  for j = 1 to n
5      C[A[j]] = C[A[j]] + 1
6  // Ora in C[i] c'è il numero di elementi uguali a i.
7  for i = 1 to k
8      C[i] = C[i] + C[i - 1]
9  // Ora in C[i] c'è il numero di elementi minori o uguali a i.
10 // Copia A in B partendo dal fondo di A.
11 for j = n downto 1
12     B[C[A[j]]] = A[j]
13     C[A[j]] = C[A[j]] - 1    // Per gestire valori duplicati
14 return B
```

L'algoritmo prende in input un'array $A[1 : n]$, la dimensione n dell'array e una soglia superiore k sui valori in A (in altre parole k è il valore massimo negli elementi dell'array), che sono interi non negativi. Restituisce l'array $B[1 : n]$ che contiene l'output ordinato e usa un array $C[0 : k]$ come memoria temporanea di lavoro.

Esempio.

Applichiamo l'algoritmo sull'array $A = [2 \ 5 \ 3 \ 0 \ 2 \ 3 \ 0 \ 3]$, il numero massimo è 5 quindi $k = 5$:



Il funzionamento della procedura Counting Sort su un array di input $A[1 : 8]$, dove ogni elemento di A è un intero non negativo non maggiore di $k = 5$.

(a) l'array A e l'array ausiliario C dopo la riga 5.

(b) l'array C dopo la riga 8

(c)-(e) l'array di output B e l'array ausiliario C dopo, rispettivamente, una, due e tre iterazioni del ciclo (righe 11-13) le caselle di colore grigio chiaro nell'array B rappresentano gli elementi che sono stati inseriti.

(f) l'array di output B è ordinato.

2.11 Bucket sort

Come il Counting Sort, il Bucket Sort è veloce perchè fa delle assunzioni sull'input. Mentre il Counting Sort suppone che l'input sia formato da interi appartenenti a un intervallo, il bucket sort suppone che l'input sia generato da un processo casuale che distribuisce gli elementi uniformemente e indipendentemente nell'intervallo $[0, 1)$.

Ciò vuol dire che l'input di array $A[1 : n]$ deve rispettare la condizione che ogni elemento $A[i]$ dell'array deve essere tale che:

$$0 \leq A[i] < 1$$

Il Bucket Sort divide l'intervallo $[0, 1)$ in n sottointervalli della stessa dimensione, detti **bucket** (cesti), e poi distribuisce gli n numeri di input nei bucket. Per produrre l'output, ordiniamo i numeri in ogni bucket (attraverso un altro algoritmo di ordinamento), che scorriamo ordinatamente, elencando gli elementi in ciascuno di essi.

Pseudocodice BucketSort

BUCKET-SORT(A, n)

```

1  sia  $B[0 : n - 1]$  un nuovo array
2  for  $i = 0$  to  $n - 1$ 
3      crea la lista vuota  $B[i]$ 
4  for  $i = 1$  to  $n$ 
5      inserisci  $A[i]$  nella lista  $B[n \cdot A[i]]$ 
6  for  $i = 0$  to  $n - 1$ 
7      ordina la lista  $B[i]$  con l'insertion sort
8  concatena ordinatamente le liste  $B[0], B[1], \dots, B[n - 1]$ 
9  return la lista concatenata
```

La supposizione che $0 \leq A[i] < 1$ però può essere limitante per problemi reali dove sono presenti numeri più grandi. Quindi, quando abbiamo valori che sono maggiori di 1, possiamo comunque usare il Bucket Sort ma bisogna **normalizzare** l'elemento dell'array prima di metterlo nei bucket. La normalizzazione avviene così:

$$\text{Valore normalizzato} = \frac{\text{Valore attuale} - \text{Valore minimo}}{\text{Valore massimo} - \text{Valore minimo}}$$

Ad esempio, se stiamo ordinando un array di valori tra 10 e 100 e stiamo analizzando il l'elemento 55, il valore normalizzato sarà:

$$\text{Valore normalizzato} = \frac{55 - 10}{100 - 10} = 0,5$$

E in base a cosa organizziamo gli elementi dell'input nei bucket?

Gli elementi vengono organizzati nei bucket calcolando in quale sottointervallo ricade il loro valore. Il numero totale di bucket da creare si sceglie spesso in base alla dimensione dell'input (n), ma la regola fondamentale è che tutti i bucket devono coprire un intervallo di valori della stessa identica ampiezza.

Infatti il Bucket Sort è particolarmente efficace quando i dati sono distribuiti in modo uniforme su un intervallo.

Riassumendo, gli step principali del Bucket Sort sono questi:

1. **Distribuzione:** gli elementi vengono distribuiti in diversi bucket in base al loro valore.
2. **Ordinamento locale:** ogni bucket viene ordinato separatamente (con un altro algoritmo di ordinamento).
3. **Concatenazione:** i bucket ordinati vengono concatenati per ottenere l'output finale.

Analisi Bucket Sort

Sia n la dimensione dell'input e k il costo per produrre l'output.

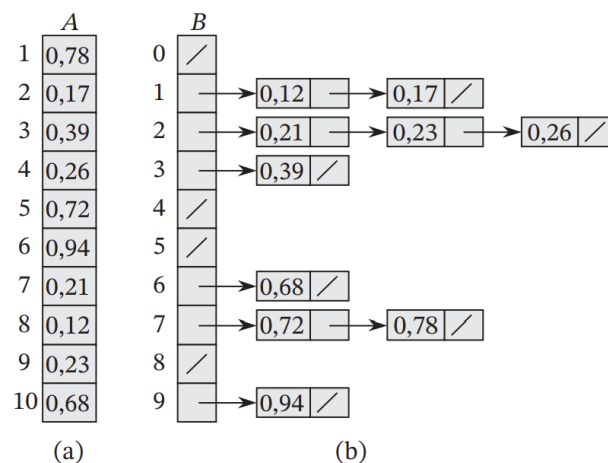
In base al caso in cui ci troviamo avremo costi temporali e spaziali differenti:

- **Caso peggiore:** avviene quando tutti gli elementi dell'input vengono messi nello stesso bucket.
 - Costo temporale: $O(n^2)$
 - Costo spaziale: $O(n + k)$
- **Caso medio:** avviene quando gli elementi dell'input si distribuiscono in maniera non perfettamente uniforme nei bucket.
 - Costo temporale: $O(n)$
 - Costo spaziale: $O(n + k)$
- **Caso ottimale:** avviene quando gli elementi dell'input vengono distribuiti in maniera perfettamente uniforme nei bucket, ovvero ogni bucket ha lo stesso numero di elementi.
 - Costo temporale: $O(n)$
 - Costo spaziale: $O(n + k)$

Esempio.

Ecco un esempio facendo riferimento allo pseudocodice di Bucket Sort.

Si vuole ordinare l'array $A = [0.78 \ 0.17 \ 0.39 \ 0.26 \ 0.72 \ 0.94 \ 0.21 \ 0.12 \ 0.23 \ 0.68]$



Il funzionamento di Bucket Sort con $n = 10$ e tutti gli elementi compresi tra 0 e 1, quindi non c'è bisogno della normalizzazione.

- (a) L'array di input $A[1 : 10]$
- (b) L'array $B[0 : 9]$ formato da liste ordinate (i bucket) dopo l'esecuzione della riga 7 dell'algoritmo, le barre "/" indicano la fine di ciascun bucket. L' i -esimo bucket contiene i valori che cadono nell'intervallo semiaperto $\left[\frac{i}{10}, \frac{(i+1)}{10}\right)$. L'output ordinato è formato dalla concatenazione ordinata delle liste $B[0], B[1], \dots, B[9]$.

2.12 Radix Sort LSD

Il Radix Sort LSD (Least Significant Decimal) è un algoritmo di ordinamento per valori numerici interi. Questo algoritmo risolve il problema dell'ordinamento dei numeri in maniera controintuitiva, ordinando prima i numeri in base alla cifra meno significativa. Concretamente, vogliamo ordinare dei numeri aventi d cifre, mettiamo in colonna i numeri e prendiamo in considerazione la colonna delle cifre meno significative (quindi la colonna tutta a destra) e ordiniamo i numeri a seconda dell'ordinamento delle cifre in quella colonna.

Vediamo un esempio numerico:

Esempio.

Dobbiamo ordinare i seguenti numeri: 329 457 657 839 436 720 355

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Il codice per il Radix Sort è semplice, l'algoritmo suppone che ogni elemento nell'array $A[1 : n]$ abbia d cifre, dove la cifra 1 è quella di ordine più basso e la cifra d è quella di ordine più alto. Ecco il codice:

```
for i = 1 to d
    usa un ordinamento stabile per ordinare l'array A[1:n]
    rispetto alla cifra i
```

È essenziale che gli ordinamenti delle cifre in questo algoritmo siano **stabili**.

Ma cosa vuol dire che un algoritmo di ordinamento deve essere stabile?

Definizione 2.12.1. (Algoritmo stabile) Un algoritmo di ordinamento è stabile se preserva le relazioni delle posizioni fornite come input.

Per capire meglio facciamo un esempio:

Esempio.

Devo ordinare i seguenti numeri: 3 1* 2 1

Nell'array ci sono due 1, quindi se l'algoritmo è stabile allora viene preservato l'ordine degli 1 e l'output sarà:

1* 1 2 3

Altrimenti l'algoritmo è considerato non stabile.

2.13 Riduzione (fra problemi)

Definizione 2.13.1. (Riduzione fra problemi) Nel campo degli algoritmi, ridurre un Problema A a un Problema B significa trasformare un'istanza del Problema A in un'istanza del Problema B , in modo da poter usare un algoritmo che risolve B per risolvere anche A .

2.14 HeapSort

Definizione Heap

Un Heap è una struttura dati, basata su alberi binari, ovvero alberi che possono avere al massimo due figli. Queste strutture possono essere rappresentate mediante array i cui elementi sono i nodi dell'albero.

proprietà

- ogni elemento dell'albero corrisponde ad un elemento dell'array
- tutti i livelli dell'albero sono completamente riempiti salvo l'ultimo che può essere parzialmente riempito da sinistra fino ad un certo punto.

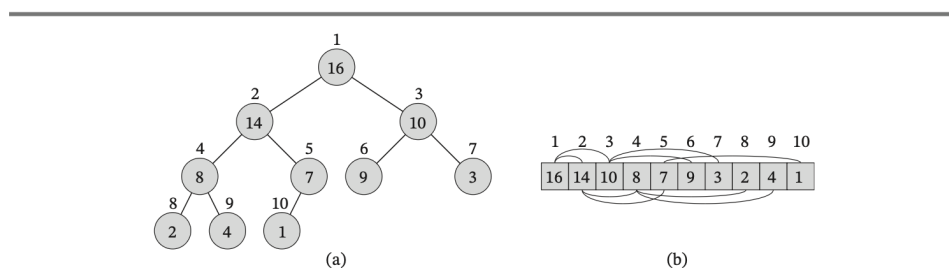


Figura 6.1 Un max-heap visto come (a) un albero binario e (b) un array. Il numero all'interno del cerchio di un nodo dell'albero è il valore memorizzato in quel nodo. Il numero sopra un nodo è il corrispondente indice dell'array. Sopra e sotto l'array ci sono delle linee che rappresentano le relazioni padre-figlio; i padri sono sempre a sinistra dei loro figli. L'albero ha altezza 3; il nodo, con indice 4 e valore 8, ha altezza 1.

Un heap è rappresentato da un oggetto che contiene

- un array di elementi $A[1..n]$
- un intero $size$ (tale che $0 \leq size \leq n$)

Un array A che rappresenta un heap ha due attributi specifici: **A.length** che indica il numero di elementi dell'array e **A.heapSize** che indica il numero di elementi dell'array che appartengono all'heap. In generale si ha che $0 \leq A.heapSize \leq A.length$.

Per ogni elemento i dell'array, ad eccezione della radice che è l'elemento $A[1]$, è possibile determinare il padre e i suoi due figli, in particolare:

$$PARENT(i) = \lfloor \frac{i}{2} \rfloor$$

$$LEFT(i) = 2i$$

$$RIGHT(i) = 2i+1$$

Max-heap e Min-heap

Esistono due tipi particolari di heap chiamati max-heap e min-heap.

Un **max-heap** è un heap in cui, per ogni nodo i diverso dalla radice, abbiamo che $A[PARENT(i)] \geq A[i]$, ovvero il padre è maggiore del figlio.

Un **min-heap** è un heap in cui, per ogni nodo i diverso dalla radice, abbiamo che $A[PARENT(i)] \leq A[i]$, ovvero i figli sono maggiori del padre.

Osservazione

In un max-heap la radice rappresenta il massimo.

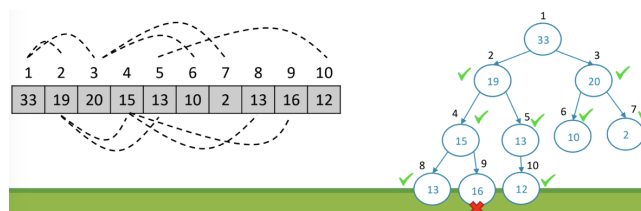
In un min-heap la radice rappresenta il minimo.

Esempio:

Dato l'array

33	19	20	15	13	10	2	13	16	12
----	----	----	----	----	----	---	----	----	----

determinare il max-heap:



Max-Heapify

L'heap memorizza elementi e può essere modificato. Per non perdere proprietà durante le modifiche della strutture dati utilizziamo l'algoritmo MAX-HEAPIFY(A,i) che si occupa di costruire un max-heap con radice nel nodo i.

pseudocodice MAX-HEAPIFY(A,i)

```

MAX-HEAPIFY(A,i)
1  l = left(i)
2  r = right(i)
3  if i <= A.heapSize AND A[l] > A[i]
4      largest = l
5  else largest = i
6  if r <= A.shapeSize AND A[r] > A[largest]
7      largest = r
8  if largest != i
9      scambia A[i] con A[largest]
10 MAX-HEAPIFY(A,largest)

```

Funzionamento

Quando viene chiamata questa funzione, si assume che i sottoalberi con radici **left(i)** e **right(i)** siano max-heap, mentre non è certo che il padre **i** sia effettivamente più grande dei figli. Vengono salvati gli indici del figlio sinistro e del figlio destro, dopodiché si procede a verificare innanzitutto se questi indici sono nei limiti dell'array e se viene rispettata la condizione $A[i] > A[l]$ e $A[r]$, se così non fosse verrebbe salvato nella variabile **largest** l'indice del figlio maggiore e verrebbero scambiati $A[largest]$ e il padre.

Così facendo la condizione di un max-heap sarebbe verificata per la terna appena considerata, tuttavia potrebbe non essere verificata nel sottoalbero che ha come radice il nodo appena scambiato, allora si procede a richiamare ricorsivamente la funzione sull'indice di largest, ovvero il nodo appena scambiato. Dunque, in breve, si fa scendere il valore $A[i]$ nel max-heap in modo che il sottoalbero con radice di indice i diventi un max-heap.

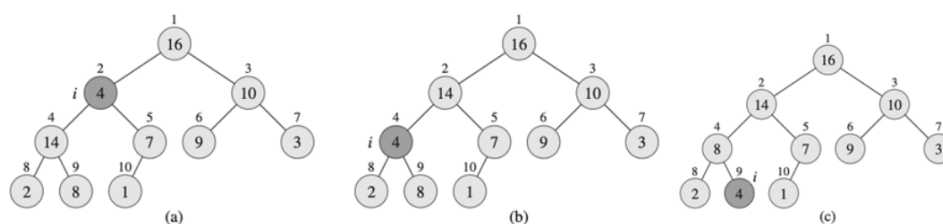


Figura 6.2 Il funzionamento di MAX-HEAPIFY(A,2), dove $A.heap-size = 10$. (a) La configurazione iniziale, con $A[2]$ nel nodo $i = 2$, che viola la proprietà di max-heap, in quanto non è più grande di entrambi i figli. In (b) la proprietà di max-heap viene ripristinata nel nodo 2 scambiando $A[2]$ con $A[4]$; ma questo distrugge la proprietà di max-heap nel nodo 4. La chiamata ricorsiva MAX-HEAPIFY(A, 4) adesso ha $i = 4$. Dopo aver scambiato $A[4]$ con $A[9]$, come mostrato in (c), il nodo 4 è sistemato e la chiamata ricorsiva MAX-HEAPIFY(A, 9) non apporta ulteriori modifiche alla struttura dati.

costo

dalla riga a 9 faccio solo confronti perciò ha costo $O(1)$, mentre la chiamata ricorsiva fatta al più h volte, dove h è l'altezza del sottoalbero ($< \log n$). Quindi ha costo **$O(h)$** .

Costruire un max-heap

La procedura Build-Max-Heap eseguita in un tempo lineare, genera da un array di input non ordinato un max-heap;

```
BUILD-MAX-HEAP(A,n)
1  A.heapSize = n
2  for i = [n/2] down to 1
3      MAX-HEAPIFY(A,i)
```

Funzionamento

La funzione Max-Heapify può essere utilizzata in modo alternativo, ovvero dal basso verso l'alto, per convertire un array di input con $n = A.length$ in un max-heap. E' possibile utilizzare la funzione Max-Heapify poiché gli ultimi elementi di ciascun sottoalbero possono essere considerati foglie dell'albero, dunque si parte da esse per salire verso la radice. Build-Max-Heap si limita a richiamare iterativamente Max-Heapify su A passando l'indice i che varia dalla metà dell'array decrementato fino ad 1, così facendo fa salire nell'albero i valori più grandi e fa scendere quelli più bassi.

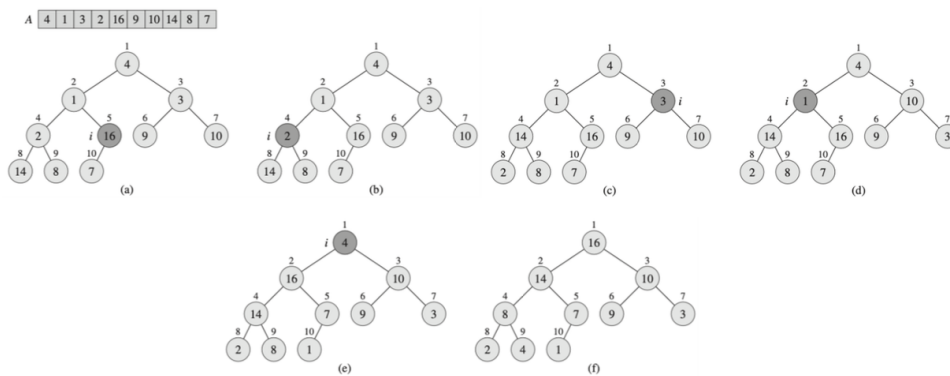


Figura 6.3 Il funzionamento di BUILD-MAX-HEAP, illustrato mostrando la struttura dati prima della chiamata della procedura MAX-HEAPIFY nella riga 3 di BUILD-MAX-HEAP. **(a)** Un array di input di 10 elementi e l'albero binario che esso rappresenta. La figura mostra che l'indice i del ciclo fa riferimento al nodo 5 prima della chiamata della procedura MAX-HEAPIFY(A, i). **(b)** La struttura dati risultante. L'indice i del ciclo per la successiva iterazione fa riferimento al nodo 4. **(c)-(e)** Le successive iterazioni del ciclo **for** in BUILD-MAX-HEAP. Notate che, ogni volta che viene chiamata la procedura MAX-HEAPIFY su un nodo, i due sottoalberi del nodo sono entrambi max-heap. **(f)** Il max-heap alla fine della procedura BUILD-MAX-HEAP.

costo

Upper bound $O(n \log n)$ perché MAX-HEAPIFY mi costa $\log n$ e lo ripeto al più n volte

Questo limite superiore sebbene corretto, non è asintoticamente stretto.

Possiamo ottenere un limite più stretto osservando che il tempo per eseguire Max-Heapify in un nodo varia con l'altezza del nodo nell'albero, e le altezze della maggior parte di essi sono più piccole. L'analisi più rigorosa si basa sulle proprietà che un heap di n elementi ha altezza $\lfloor \log n \rfloor$ e, per ogni h , ha al più $\lceil n/2^{h+1} \rceil$ nodi di altezza h . Il tempo richiesto dalla procedura Max-Heapify quando viene chiamata per un nodo di altezza h è $O(h)$.

Possiamo dire che il costo totale di build-Max-Heap è limitato superiormente da

$$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil O(h) = O(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h})$$

$$\text{con } \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \rightarrow \sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$$

Perciò ho $O(n)$.

Correttezza di Build-Max-Heap

Per verificare che Build-Max-Heap funziona correttamente, utilizziamo la seguente invariante di ciclo.

All'inizio di ogni iterazione del ciclo **for**, righe 2-3, ognuno dei nodi $i+1, i+2, \dots, n$ è la radice di un max-heap

Inizializzazione

Prima della prima iterazione del ciclo, $i = \lfloor n/2 \rfloor$

. Ognuno dei nodi $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ è una foglia e, quindi, è la radice di un max-heap con un solo nodo.

Mantenimento

per verificare che ogni iterazione conserva l'invariante di ciclo, notiamo che i figli del nodo i hanno una numerazione più alta di i . Per l'invariante di ciclo, quindi, essi sono entrambi radici di un max-heap. Questa è esattamente la condizione richiesta affinché la chiamata $\text{Max-Heapify}(A,i)$ renda il nodo i la radice di un max-heap. Inoltre, la chiamata di Max-Heapify preserva la proprietà che tutti i nodi $i+1, i+2, \dots, n$ siano radici di un max-heap. Il fatto che i venga decrementato di un'unità a ogni iterazione del ciclo for ristabilisce l'invariante di ciclo per la successiva iterazione.

Conclusione

Il ciclo esegue esattamente $\lfloor n/2 \rfloor$ iterazioni, e quindi termina. Alla fine del ciclo si ha $i = 0$. Per l'invariante di ciclo, ognuno dei nodi $1, 2, \dots, n$ è la radice di un max-heap; in particolare, lo è il nodo 1.

Heap Sort

Che eseguita in un tempo $O(n \log n)$, ordina un array.

$\text{HeapSort}(A, n)$

```

1  BUILD-MAX-HEAP(A, n)
2  for i = n down to 2
3      scambia A[1] con A[i]
4      A.heapSize = A.heapSize - 1
5      MAX-HEAPIFY(A, 1)
```

Funzionamento

La funzione HeapSort chiama inizialmente $\text{Build-Max-Heap}(A, n)$ per trasformare A in un max-heap. Dunque, se per definizione l'elemento più grande dell'array è memorizzato nella radice $A[1]$, per ordinare in modo crescente l'albero, basta scambiare la radice con l'elemento in posizione finale, ovvero $A[n]$. Dopodiché si decrementa $A.\text{heapSize}$, in modo tale da non considerare più l'elemento in ultima posizione, in quanto si trova nella sua posizione definitiva. Per ottenere un max-heap basta chiamare $\text{Max-Heapify}(A, 1)$ per far tornare il max alla radice. Questo ciclo viene ripetuto $n-1$ volte. Build-Max-Heap impiega $O(n)$, mentre Max-Heapify $O(\log n)$, in totale il costo di HeapSort ammonta a $O(n \log n)$.

Capitolo 3

Il problema della selezione e le statistiche d'ordine

Definizione 3.0.1. Statistica d'ordine

Dato un insieme A di n numeri (distinti), la k -esima statistica d'ordine è il k -esimo elemento più piccolo di A .

- il minimo di un insieme A è la prima statistica d'ordine.
- il massimo di un insieme A è l' n -esima statistica d'ordine.
- la mediana di un insieme A è l'elemento di mezzo.
 - se dispari una mediana $\lfloor (n+1)/2 \rfloor$.
 - se pari due mediana una inferiore e una superiore $\lceil (n+1)/2 \rceil$.

3.1 Problema selezione

Dato un insieme di elementi A il problema della selezione consiste nel selezionare la k -esima statistica d'ordine. definiamo il **problema della selezione** come segue: **input:** un insieme A di n numeri (distinti) ed un intero k , con $1 \leq k \leq n$. **output:** $x \in A$ tale che x è il k -esimo in ordine di grandezza.

osservazione:

dato un elemento $x \in A$, x è il k -esimo in ordine di grandezza se gli altri $n-1$ elementi si possono ripartire in 2 gruppi tali che:

- un gruppo contiene $k-1$ elementi tutti minori o uguali di x ed
- un gruppo contiene $n-k$ elementi tutti maggiori o uguali di x

3.2 Minimo o Massimo

Trovare il minimo e il massimo di un insieme sono problemi molto comuni e per cui già conosciamo un algoritmo semplice:

```
1  Minium(a,n)
2      min=A[1]
3      for i=2 to n
4          if min > A[i]
5              min = A[i]
6      return min
```

osservazioni $n-1$ confronti è un upper bound per il problema del minimo/massimo
L'algoritmo proposto è ottimo rispetto al numero di confronti e la complessità è $\Theta(n)$

3.3 Minimo e Massimo

Come sappiamo per cercare il min o il max siamo abituati a svolgere n confronti, ovvero confrontare ciascun elemento dell'array con il min e max.

Per trovare sia il minimo che il massimo, eseguo in sequenza Minium e il duale Maxium (soluzione Naif) e ottengo un totale di $2(n-1)$ confronti. Il costo asintotico nel caso peggiore per la ricerca del min e del max è $\Theta(n)$, non è possibile trovare un algoritmo che faccia meglio. Nonostante ciò, è comunque possibile scansionarli a coppie così che il numero di cicli passi da n a $\frac{n}{2}$ e con al più $3\lfloor n/2 \rfloor$.

Psudocodice

```

1  Min-Max(A,n)
2  if (n è dispari) then
3      max=A[1];
4      min=A[1];
5      i=2;
6  else
7      if(A[1]<A[2]) then
8          max=A[2];
9          min=A[1];
10         i=3;
11     else
12         max=A[1];
13         min=A[2];
14         i=3;
15     end
16 end
17 while(i<n) do
18     if(A[i]<A[i+1]) then
19         if (A[i]<min) then
20             min=A[i];
21         end
22         if (A[i+1]>max) then
23             max=A[i+1];
24         end
25     else
26         if(A[i+1]<min) then
27             min=A[i+1];
28         end
29         if(A[i]>max) then
30             max=A[i];
31         end
32     end
33     i=i+2;
34 end
35 return min,max

```

Prima confrontiamo due elementi di input l'uno con l'altro e poi il più piccolo dei due con il minimo corrente, e il più grande dei due con il massimo corrente, con un costo di 3 confronti ogni 2 elementi.

Come si debba impostare i valori iniziali del minimo e del massimo dipende dal fatto che n sia pari o dispari. Se n è dispari, assegniamo al minimo e al massimo il valore del primo elemento, e poi esaminiamo i restanti elementi a coppie. Se n è pari, effettuiamo un confronto fra i primi 2 elementi per determinare i valori iniziali del minimo e del massimo; poi esaminiamo i restanti elementi a coppie, come nel caso di n dispari.

Nelle iterazioni del ciclo while vengono effettuati sempre 3 confronti, allora è possibile dire che il numero di confronti se n è pari o dispari è:

$$n \text{ dispari: } 3 \cdot \left(\frac{n-1}{2}\right) \quad n \text{ pari: } 3 \cdot \left(\frac{n-2}{2}\right) + 1$$

Nel caso di n dispari viene assegnato ad i il valore 2, questo significa che il while farà $\frac{n-1}{2}$ cicli; infatti, bisogna notare come i venga incrementato di 2 ogni volta, dimezzando così il numero di cicli. Il tutto è moltiplicato per 3 in quanto come abbiamo visto il numero di confronti per il ciclo è 3. Nel caso n pari dobbiamo considerare il costo costante del confronto (+1) e notare in seguito come ad i venga assegnato il valore 3. Questo significa che, ragionando come prima, il numero di cicli che effettuerà il while sarà $\frac{n-2}{2}$, il tutto moltiplicato per 3 dato il numero di confronti per ciclo.

Esempio.

$A = \langle 5, 8, 23, 1, 7, 9 \rangle$ con $n = 6$

Passo 1:

n è pari allora prendo i primi due valori dell'array e li confronto e assegno $min = 5$ e $max = 8$ e pongo $i = 3$

Passo 2:**Iterazione 1:**

1. prendo la coppia $A[i] = 23$ e $A[i + 1] = 1$:
2. $A[i] < A[i + 1]$? No, allora:
3. $A[i + 1] < min = 5$? Si, allora $min = 1$
4. $A[i] > max = 8$? Si, allora $max = 23$
5. $i = 3 + 2 = 5$

Iterazione 2:

1. prendo la coppia $A[i] = 7$ e $A[i+1] = 9$:
2. $A[i] < A[i + 1]$? Si, allora:
3. $A[i] < min = 1$? No
4. $A[i + 1] > max = 23$? No
5. $i = 5 + 2 = 7$
6. esce dal ciclo e il programma termina con $min = 1$ e $max = 23$.

Analisi esempio.

	n dispari	n pari
Init	1 confronto → 0 confronti	2 confronti → 1 confronto
Iterazione 1	confronti	3 confronti
Iterazione 2	3 confronti	3 confronti

3.4 Selezione Randomizzata

Il problema generale della selezione e il problema del minimo o del massimo hanno lo stesso tempo di esecuzione asintotico: $\Theta(n)$. Sfruttiamo la proprietà della funzione partizione usata nel QuickSort e continuo a cercare solo se il pivot non è la statistica d'ordine che cercavo.
in dettaglio:

- usiamo la versione modificata della procedura usata in QuickSort in cui il pivot viene scelto in modo casuale
- partizioniamo l'input come facciamo in QuickSort
- controllo in che posizione cade il pivot dopo la partizione
 - Se il pivot è nella posizione i corrispondente alla statistica d'ordine che cercavo ho finito
 - Altrimenti faccio una chiamata ricorsiva su una sola delle due partizioni
 - Quale partizione scegliere per fare la chiamata ricorsiva dipende dalla relazione tra il pivot trovato e la mia statistica d'ordine cercata

Pseudocodice

```

1 Randomized_Select(A,p,r,i)
2 if(p == r) then
3     return A[p]
4 end
5 q = Randomized_Partition(A,p,r)
6 k = q - p + 1
7 if(i == k)
8     return A[q]
9 else
10    if (i < k) then
11        return Randomized_Select(A,p,q-1,i);
12    else
13        return Randomized_Select(A,q+1,r,i-k);
14    end
15 end

16 Randomized_Partition(A,p,r)
17 i = random(p,r);
18 scambia(A[r],A[i]);
19 return partiziona(A,p,r) //stessa procedura usata in QuickSort

```

Esempio.

		p	r	i	partizionamento	utile?																															
$A^{(0)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>6</td><td>19</td><td>4</td><td>12</td><td>14</td><td>9</td><td>15</td><td>7</td><td>8</td><td>11</td><td>3</td><td>13</td><td>2</td><td>5</td><td>10</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	6	19	4	12	14	9	15	7	8	11	3	13	2	5	10	1	15	5			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
6	19	4	12	14	9	15	7	8	11	3	13	2	5	10																							
$A^{(1)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>6</td><td>4</td><td>12</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>3</td><td>13</td><td>2</td><td>5</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	6	4	12	10	9	7	8	11	3	13	2	5	14	19	15	1	12	5	1	no	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
6	4	12	10	9	7	8	11	3	13	2	5	14	19	15																							
$A^{(2)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>6</td><td>13</td><td>5</td><td>12</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	10	9	7	8	11	6	13	5	12	14	19	15	4	12	2	2	sì	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	10	9	7	8	11	6	13	5	12	14	19	15																							
$A^{(3)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>10</td><td>9</td><td>7</td><td>8</td><td>11</td><td>6</td><td>12</td><td>5</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	10	9	7	8	11	6	12	5	13	14	19	15	4	11	2	3	no	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	10	9	7	8	11	6	12	5	13	14	19	15																							
$A^{(4)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>11</td><td>9</td><td>12</td><td>10</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	5	6	7	8	11	9	12	10	13	14	19	15	4	5	2	4	sì	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	5	6	7	8	11	9	12	10	13	14	19	15																							
$A^{(5)}$	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td></tr><tr><td>3</td><td>2</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>11</td><td>9</td><td>12</td><td>10</td><td>13</td><td>14</td><td>19</td><td>15</td></tr></table>	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	3	2	4	5	6	7	8	11	9	12	10	13	14	19	15	5	5	1	5	sì	
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15																							
3	2	4	5	6	7	8	11	9	12	10	13	14	19	15																							

Figura 9.1 Il funzionamento della procedura RANDOMIZED-SELECT, rappresentata tramite successive partizioni che riducono la dimensione del sottoarray $A[p:r]$, mostrando i valori dei parametri p , r e i a ogni chiamata ricorsiva. A ogni passo della ricorsione, il sottoarray $A[p:r]$ è mostrato in grigio chiaro, mentre l'elemento più scuro è il pivot individuato per il partizionamento successivo. Gli elementi in grigio intermedio sono al di fuori di $A[p:r]$. La risposta è l'elemento più scuro dell'array più in basso, con $p = r = 5$ e $i = 1$. I nomi $A^{(0)}, A^{(1)}, \dots, A^{(5)}$, con cui sono designati i vari array, i numeri di partizionamento e il fatto che il partizionamento sia utile o no, sono spiegati nel testo.

Analisi del costo

Questo algoritmo ha un costo di caso peggiore $\Theta(n^2)$. poichè il partizionamento richiede un tempo $\Theta(n)$, la ricorrenza per il tempo di esecuzione nel caso peggiore è la stessa di quella per QuickSort, ovvero $T(n) = T(n-1) + \Theta(n)$, che ha soluzione $T(n) = \Theta(n^2)$. Tuttavia la randomizzazione mi aiuta nel caso medio in cui il tempo atteso di esecuzione è lineare.

Analisi del valore atteso del tempo di esecuzione



Supponiamo che ogni volta che l'algoritmo seleziona casualmente un pivot, questo si trovi da qualche parte tra il secondo e il terzo quartile (parte centrale, quella in verde) degli elementi rimanenti, se presi in ordine. Se l' i -esimo elemento più piccolo è minore del pivot, tutti gli elementi maggiori del pivot vengono ignorati in tutte le future chiamate ricorsive. Tra questi elementi che verranno sempre ignorati, ci sono senz'altro quelli del quartile superiore, e possibilmente anche qualcun altro. Allo stesso modo, se l' i -esimo elemento più piccolo è maggiore del pivot, tutti gli elementi minori del pivot, cioè almeno tutti quelli nel primo quartile, vengono ignorati in tutte le future chiamate ricorsive.

In entrambi i casi almeno $\frac{1}{4}$ degli elementi rimanenti viene ignorato in tutte le future chiamate ricorsive, lasciando in gioco al massimo $\frac{3}{4}$ degli elementi rimanenti, che rimarranno nell'array $A[p : r]$. Poiché Randomized-Partition richiede un tempo $\Theta(n)$ su un sottoarray di n elementi, la ricorrenza per il tempo di esecuzione nel caso peggiore è $T(n) = T(\frac{3n}{4}) + \Theta(n)$; questa ricorrenza ha soluzione $T(n) = \Theta(n)$.

naturalmente il pivot non cade necessariamente ogni volta nella metà centrale. Poiché il pivot è scelto a caso, la probabilità che cada nella metà centrale è circa $\frac{1}{2}$ ogni volta.

Ci aspettiamo che metà delle partizioni riduca il numero di elementi ancora in gioco di un numero che è almeno $\frac{1}{4}$ di quelli in gioco al passo precedente, mentre che l'altra metà delle partizioni non sia altrettanto utile. Di conseguenza il numero previsto di partizioni al massimo raddoppia, rispetto al caso in cui il pivot cade sempre nella metà centrale. Il costo di ogni partizione aggiuntiva è inferiore a quello della precedente, per cui il tempo di esecuzione atteso è ancora $\Theta(n)$.

Lemma 3.4.1. Una partizione è utile con probabilità almeno $\frac{1}{2}$.

(da correggere la frazione con la parte intera superiore)

Dimostrazione. Definiamo più precisamente la metà centrale di un array di n elementi. Questa corrisponde a tutti gli elementi tranne gli $\lceil \frac{n}{4} \rceil - 1$ più piccoli e gli $\lceil \frac{n}{4} \rceil - 1$ più grandi.

Indipendentemente dal punto in cui cade il pivot, o tutti gli elementi maggiori o tutti quelli minori, compreso stesso, non saranno più in gioco dopo la partizione. Se il pivot cade nella metà centrale, quindi, almeno gli $\lceil \frac{n}{4} \rceil - 1$ elementi minori o gli $\lceil \frac{n}{4} \rceil - 1$ elementi maggiori del pivot, esso compreso, non saranno più in gioco. Il numero di elementi che rimangono in gioco sarà al massimo $n - \lceil \frac{n}{4} \rceil$ che è uguale a $\lfloor \frac{3n}{4} \rfloor$. Poiché $\lfloor \frac{3n}{4} \rfloor \leq \frac{3n}{4}$, la partizione è utile.

Per determinare un limite inferiore alla probabilità che un pivot scelto a caso cada nella metà centrale, determiniamo un limite superiore alla probabilità che non ci cada. Tale probabilità è

$$\begin{aligned} \frac{2(\lceil n/4 \rceil - 1)}{n} &\leq \frac{2(\lceil n/4 + 1 \rceil - 1)}{n} \\ &= \frac{n/2}{n} \\ &= \frac{1}{2} \end{aligned}$$

pertanto il pivot ha probabilità almeno $\frac{1}{2}$ di cadere nella metà centrale, e quindi la probabilità che una partizione sia utile è almeno $\frac{1}{2}$. $\square$

Teorema 3.4.1. La procedura Randomized-Select ha un tempo di esecuzione atteso $\Theta(n)$ su un array di input di n valori distinti

Dimostrazione. Poiché non tutte le partizioni non sono necessariamente utili, assegniamo a ciascuna un indice a partire da 0 e indichiamo con $\langle h_0, h_1, h_2, \dots, h_m \rangle$ la sequenza di partizioni utili, in modo che la h_k -esima partizione sia utile per $k = 0, 1, 2, \dots, m$. Assumiamo convenzionalmente che la partizione fittizia di indice 0 è utile, così che $h_0 = 0$. Indichiamo $|A^{(h_k)}|$ con n_k , dove $n_0 = |A^{(0)}|$ è la dimensione del problema originale. Poiché la h_k -esima partizione è utile e le cardinalità degli insiemi $|A^{(j)}|$ decrescono strettamente, si ha $n_k = |A^{(h_k)}| \leq (3/4)|A^{(h_{k-1})}| = (3/4)n_{k-1}$ per $k = 1, 2, \dots, m$. Iterando la disuguaglianza $n_k \leq (3/4)n_{k-1}$, si ha $n_k \leq (3/4)^k n_0$ per $k = 0, 1, 2, \dots, m$.

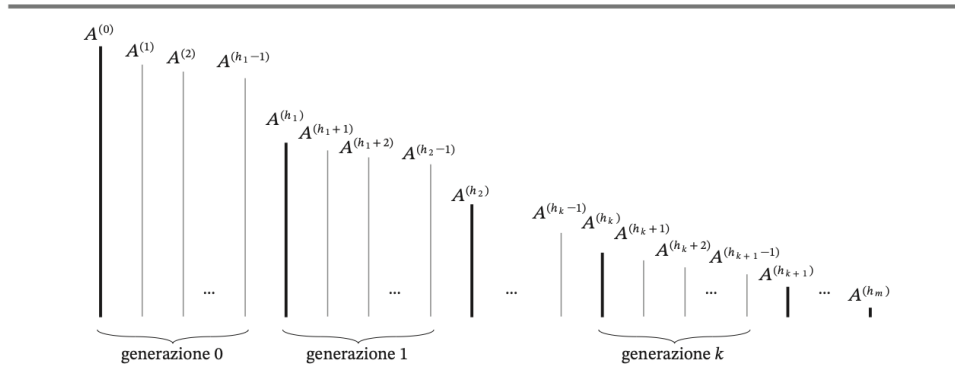


Figura 9.2 Gli insiemi all'interno di ogni generazione nella dimostrazione del Teorema 9.2. Le linee verticali rappresentano gli insiemi, e l'altezza di ogni linea rappresenta la cardinalità dell'insieme, che equivale al numero di elementi in gioco. Ogni generazione inizia con un insieme $A^{(h_k)}$, che è il risultato di una partizione utile. Questi insiemi sono rappresentati da una linea più spessa e hanno al massimo $3/4$ degli elementi degli insiemi alla loro immediata sinistra. Gli insiemi rappresentati da una linea più sottile non sono i primi di una generazione. Una generazione può anche contenere un solo insieme. Gli insiemi della generazione k sono $A^{(h_k)}, A^{(h_k+1)}, \dots, A^{(h_k+X_k-1)}$. Gli insiemi $A^{(h_k)}$ sono definiti in modo che $|A^{(h_k)}| \leq (3/4)|A^{(h_{k-1})}|$. Se la sequenza di partizioni arriva fino alla generazione m , l'insieme $A^{(h_m)}$ ha al più un solo elemento in gioco.

Ora definiamo la variabile casuale

$$X_k = h_{k+1} - h_k$$

per $k=0,1,2,\dots,m-1$. Quindi, X_k è il numero di insiemi nella k -esima generazione, di modo che gli insiemi nella k -esima generazione sono $A^{(h_k)}, A^{(h_k+1)}, \dots, A^{(h_k+X_k-1)}$.

Per il lemma precedente, la probabilità che una partizione sia utile è almeno $\frac{1}{2}$. La probabilità è in realtà maggiore, poiché una partizione è utile anche se il pivot non cade nella metà centrale, ma l' i -esimo elemento più piccolo si torva nella sezione più piccola della partizione. Utilizzeremo comunque la stima inferiore di $\frac{1}{2}$, quindi dall'equazione segue $E[X_k] \leq 2$ per $k = 0, 1, 2, \dots, m-1$.

Ne segue che l'equazione di ricorrenza calcolata in precedenza viene al massimo raddoppiata non andando ad impattare sul costo asintotico atteso.

Stimiamo superiormente il numero totale di confronti durante tutte le partizioni. Alla j -esima partizione si effettuano meno di $|A^{(j-1)}|$ confronti.

Raggruppando per generazioni, otteniamo:

$$\sum_{k=0}^{m-1} \sum_{j=h_k}^{h_{k+1}-1} |A^{(j)}| \leq \sum_{k=0}^{m-1} X_k |A^{(h_k)}|.$$

Poiché la dimensione del problema diminuisce geometricamente:

$$|A^{(h_k)}| \leq \left(\frac{3}{4}\right)^k n_0,$$

segue:

$$\sum_{k=0}^{m-1} X_k |A^{(h_k)}| \leq \sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0.$$

Prendendo il valore atteso e usando la linearità:

$$\mathbb{E} \left[\sum_{k=0}^{m-1} X_k \left(\frac{3}{4}\right)^k n_0 \right] = n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k \mathbb{E}[X_k] \leq 2n_0 \sum_{k=0}^{m-1} \left(\frac{3}{4}\right)^k.$$

Estendendo la somma a infinito:

$$< 2n_0 \sum_{k=0}^{\infty} \left(\frac{3}{4}\right)^k = 8n_0.$$

Quindi il numero atteso di confronti è $O(n)$; inoltre, essendo necessario esaminare tutti gli elementi almeno una volta, si ha anche un limite inferiore $\Omega(n)$. Pertanto, il tempo di esecuzione atteso di RANDOMIZED-SELECT è:

$$\Theta(n).$$

□

3.5 Selezione in tempo

Esamiamo ora un algoritmo di selezione interessante dal punto di vista teorico, il cui tempo di esecuzione è $\Theta(n)$ nel caso peggiore.

Come **RANDOMIZED-SELECT**, che ha un tempo di esecuzione lineare nel caso peggiore, trova l'elemento desiderato partizionando ricorsivamente l'array di input. A differenza di **RANDOMIZED-SELECT**, però, **SELECT** garantisce che la partizione sia buona scegliendo, durante la partizione dell'array, un pivot di cui si può provare la bontà. Pertanto ci sono due chiamate di **SELECT**: una per trovare un buon pivot e una seconda per trovare ricorsivamente la statistica d'ordine.

Pseudocodice

prende in input un array $A[p : r]$ di $n = r - p + 1$ elementi e un intero i tale che $1 \leq i \leq n$

```
1  Select (A,p,r,i)
2  while(r-p+1) mod5 != 0 // operazione di modulo ovvero il resto
3      for j = p+1 to r do
4          if(A[p] > A[j]) then
5              scambia(A[p],A[j]);
6          end
7      end
8      if (i == 1) then
9          return A[p]
10     end
11     p = p+1;
12     i = i-1;
13 end
14 g=(r-p+1)/5;
15 for j = p to p + g - 1 do
16     ordina_sul_posto(A[j],A[j+g],A[j+2g],A[j+3g],A[j+4g]);
17 end
18 // Cerca ricorsivamente il pivot x come mediana delle mediane
19 x = select(A,p+2g,p+3g-1,(g+1)/2);
20 q = partition_around(A,p,r,x) //partiziona intorno al pivot
21 k = q-p+1
22 if(i==k) then
23     return A[q];
24 else
25     if(i<k) then
26         return Select(A,p,q-1,i);
27     else
28         return Select(A,q+1,r,i-k);
29     end
30 end
```

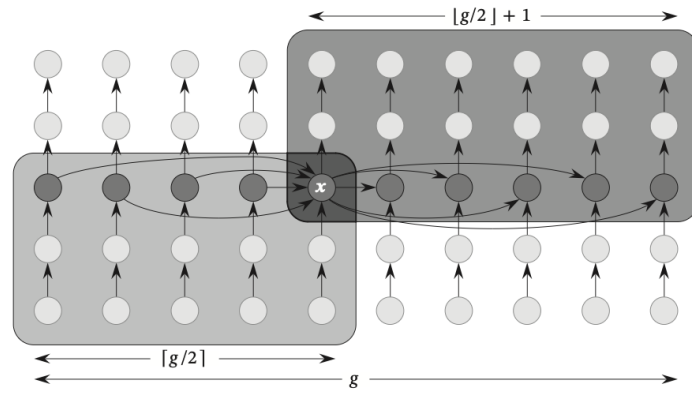


Figura 9.3 Le relazioni tra gli elementi di A (rappresentati da cerchi) subito dopo la riga 17 dell'algoritmo di selezione SELECT. Ci sono $g = (r - p + 1)/5$ cinquine di elementi, ognuna delle quali occupa una colonna. Per esempio, la colonna più a sinistra contiene gli elementi $A[p], A[p + g], A[p + 2g], A[p + 3g], A[p + 4g]$ e la colonna successiva contiene $A[p + 1], A[p + g + 1], A[p + 2g + 1], A[p + 3g + 1], A[p + 4g + 1]$. Le mediane dei gruppi sono i cerchi scuri e il pivot x è indicato con una etichetta. Le frecce vanno dagli elementi più piccoli a quelli più grandi. Gli elementi su sfondo grigio in basso a sinistra sono tutti minori o uguali a x e non possono cadere nella sezione alta della partizione fatta intorno a x . Gli elementi su sfondo grigio in alto a destra sono maggiori o uguali a x e non possono cadere nella sezione bassa della partizione fatta intorno a x . Il pivot x appartiene a entrambe le regioni con sfondo grigio ed è mostrato su uno sfondo più scuro. Gli elementi su sfondo bianco potrebbero trovarsi in entrambe le sezioni della partizione.

Le cinquine sono costruite verticalmente, in modo che gli elementi che le costruiscono siano vicini tra loro. Di ciascuna si seleziona la mediana, con la caratteristica di essere maggiore degli elementi precedenti e minore dei successivi della cinquina. In questo modo si è messo in evidenza l'insieme delle mediane. Con l'istruzione $x = \text{select}(A, p + 2g, p + 3g - 1, (g + 1)/2)$; si cerca la mediana delle mediane all'interno dell'insieme, questo coincide con l'elemento al centro dell'array complessivo. Una volta chiamata la funzione partiziona questa sposta tutti gli elementi minori di x nel sottoarray di sinistra e quelli maggiori nel sottoarray di destra. Se $i == k$ si è trovata la k -esima statistica d'ordine, altrimenti si chiama ricorsivamente la funzione select sul sottoarray di sinistra se $i < k$, o su quello di destra.

Capitolo 4

Programmazione Dinamica

4.1 Panoramica generale

La programmazione dinamica, come il metodo Divide et Impera, risolve i problemi combinando le soluzioni dei sottoproblemi. La programmazione dinamica può essere applicata quando i sottoproblemi non sono indipendenti, ovvero quando i sottoproblemi hanno in comune dei sottoproblemi.

A differenza del metodo Divide et Impera, la programmazione dinamica risolve ciascun sotto - sottoproblema **una sola volta** e salva la sua soluzione in una tabella, evitando così di ricalcolare la soluzione ogni volta che si presenta il sotto - sottoproblema.

La programmazione dinamica, tipicamente, si applica ai **problemi di ottimizzazione**. Per questi problemi ci possono essere molte soluzioni possibili, ogni soluzione ha un valore e vogliamo trovare una soluzione con il valore ottimo.

4.2 Sviluppo dell'algoritmo

Per sviluppare un algoritmo che faccia uso della programmazione dinamica, seguiamo una strategia formata dalle seguenti quattro fasi:

1. Identificare i sottoproblemi.
2. Definire la funzione di stato.
3. Scegliere tra un approccio TOP DOWN o BOTTOM UP.
4. Recuperare la soluzione (se necessario).

Vediamo nel dettaglio i vari passaggi.

Funzione di stato

Dopo aver identificato i sottoproblemi dobbiamo definire la **funzione di stato**.

Definizione 4.2.1. (Funzione di Stato) La funzione di stato è la funzione che descrive la soluzione di un sottoproblema.

Ogni sottoproblema è identificato da una o più variabili di stato, cioè i parametri che descrivono in quale punto del problema ci si trova. La funzione di stato permette di tradurre il problema in una forma ricorsiva.

Ora vediamo i due approcci TOP DOWN e BOTTOM UP:

Top - Down

Con l'approccio **Top Down** con **memoizzazione** scriviamo la soluzione ricorsiva, salvando il risultato di ciascun sottoproblema (di solito in un array o in una tabella hash). La procedura prima verifica se ha risolto precedentemente questo sottoproblema. In caso affermativo, restituisce il valore salvato, risparmiando gli ulteriori calcoli a quel livello; altrimenti, la procedura calcola il valore nel modo usuale e lo memoizza. Diciamo che la procedura ricorsiva è stata dotata di un blocco note (memo) e quindi memoizzata, nel senso che “annota” quali risultati sono stati calcolati in precedenza.

Bottom Up

Con l'approccio **Bottom Up** Risolviamo i problemi in ordine di grandezza, partendo dai più piccoli, e tenendo da parte la soluzione di ogni sottoproblema la prima volta che viene incontrato. Così facendo, quando si risolve un certo sottoproblema, le soluzioni di tutti quelli più piccoli da cui dipende sono già state salvate. Dobbiamo risolvere un sottoproblema una sola volta e la prima volta in cui ci imbattiamo in esso abbiamo già risolto tutti i sottoproblemi da cui dipende la sua soluzione.

Esempio.

Prendiamo in considerazione il problema della successione di **Fibonacci**.

Per prima cosa vediamo un codice che non segue le regole della programmazione dinamica:

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
// Complessità:  $O(2^n)$ 
```

Questa soluzione segue il metodo Divide et Impera, ma è molto inefficiente poichè sottoproblemi uguali vengono calcolati più volte, senza alcuna memorizzazione.

Ora utilizziamo la programmazione dinamica.

Vediamo prima l'approccio TOP DOWN con memoizzazione:

```
#define MAX 100
int memo[MAX];
int fib_memo(int n) {
    if (memo[n] != -1) return memo[n];
    if (n <= 1) return n;
    memo[n] = fib_memo(n-1) + fib_memo(n-2);
    return memo[n];
}
// In main(): inizializzare memo con -1
```

In questa soluzione ogni valore delle chiamate ricorsive viene salvato in `memo[n]`, in questo modo non servirà ricalcolare sottoproblemi già calcolati in precedenza.

Infine vediamo l'approccio BOTTOM UP:

```
int fib_tab(int n) {
    int dp[MAX];
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Qui vengono prima calcolati i primi due sottoproblemi (`dp[0]` `dp[1]`), che poi verranno usati nel ciclo `for` e non dovranno essere ricalcolati.

4.3 Problema dello zaino 0/1

Immaginiamo di possedere uno zaino che può sopportare un peso massimo W . Abbiamo a disposizione n oggetti, ogni oggetto ha:

- Un **peso** (`peso[i]`)
- Un **valore** (`valore[i]`)

L'**obiettivo** è massimizzare il valore totale degli oggetti che mettiamo nello zaino, senza che il loro peso totale superi il valore di peso massimo w . Si chiama (0/1) perchè ogni oggetto o lo prendiamo intero (1) o lo lasciamo (0), non è possibile quindi prendere frazioni di oggetti.

Questo è il codice che risolve il problema dello zaino 0/1:

```
int knapsack(int n, int W, int peso[], int valore[]) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (peso[i-1] <= w)
                dp[i][w] = fmax(dp[i-1][w], dp[i-1][w - peso[i-1]] + valore[i-1]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}
```

Ecco la spiegazione della soluzione proposta:

1. `dp[n+1][W+1];`
Viene introdotta la **funzione di stato**, che rappresenta il massimo valore con n oggetti e capacità massima W .
2. `for (int i = 0; i <= n; i++)`
Inizia il ciclo esterno, iteriamo sul numero di oggetti a nostra disposizione.
3. `for (int w = 0; w <= W; w++)`
Inizia il ciclo interno, per ogni numero di oggetti i , proviamo a riempire zaini di capacità crescente, da 0 fino alla capacità massima W .
4. `if (i == 0 || w == 0) dp[i][w] = 0;`
Con questa condizione viene implementato il **caso base**, se stiamo considerando 0 oggetti o abbiamo una capacità massima nulla, impostiamo il valore a 0.
5. `else if (peso[i-1] <= w);`
Con questa condizione si verifica se l'oggetto corrente ($i-1$) può entrare nello zaino senza superare il peso disponibile w . Se l'oggetto non può entrare perchè troppo pesante, allora `dp[i][w] = dp[i-1][w];`, ovvero copiamo semplicemente il valore che avevamo ottenuto senza quell'oggetto.
6. `dp[i][w] = fmax(dp[i-1][w], dp[i-1][w - peso[i-1]] + valore[i-1]);`
Se l'oggetto può entrare, devo fare una scelta che massimizzi il mio guadagno. Usiamo la funzione `fmax` per decidere se conviene di più:
 - Lasciare l'oggetto (`dp[i-1][w]`)
 - Prendere l'oggetto (`dp[i-1][w - peso[i-1]] + valore[i-1]`)

Qui l'uso degli indici può sembrare difficile da capire, perciò vediamo nel dettaglio cosa vogliono dire quelle due espressioni. Se lasciamo l'oggetto corrente, impostiamo che `d[i][w] = dp[i-1][w]`, quindi stiamo attribuendo a `d[i][w]` un valore che già era stato calcolato in precedenza e che è stato ritenuto il valore massimo dalla funzione `fmax` rispetto al valore che avremmo ottenuto prendendo l'oggetto. Siccome non prendiamo l'oggetto l'indice w rimane invariato, in quanto il peso corrente non cambia.

Invece se prendiamo l'oggetto, impostiamo che `d[i][w] = dp[i-1][w - peso[i-1]] + valore[i-1]`. Siccome stiamo prendendo l'oggetto, dobbiamo considerare due cose: la prima è aggiungere il valore dell'oggetto che prendiamo, e questo lo facciamo sommando `valore[i-1]`, la seconda è che ora il peso disponibile diminuisce visto che abbiamo preso l'oggetto e quindi dobbiamo sottrarre alla capacità corrente (w) il peso dell'oggetto che prendiamo (`peso[i-1]`), quindi scriviamo `w - peso[i-1]`.

7. `return dp[n][W];`
Restituisco il risultato finale: il valore massimo ottenibile considerando tutti gli n oggetti e sfruttando tutta la capacità W dello zaino.

4.4 Cammini su griglia (senza ostacoli)

Data una griglia rettangolare $m \times n$ si ha che:

- **Partenza:** si parte alle coordinate $0, 0$
- **Arrivo:** si deve raggiungere la cella in basso a destra, alle coordinate $(m - 1, n - 1)$
- **Vincoli:** ci si può muovere solo verso destra oppure verso il basso

L'**obiettivo** è quello di contare quanti percorsi diversi (distinti) esistono per andare dalla partenza all'arrivo. Seguiamo la logica della programmazione dinamica e spezziamo il problema in sottoproblemi. Quindi piuttosto che cercare di calcolare tutti i percorsi fino all'arrivo in un colpo solo, cerchiamo di capire quanti percorsi ci sono per arrivare a una singola cella intermedia.

Questo è il codice che risolve il problema:

```
int unique_paths(int m, int n) {
    int dp[m][n];
    for (int i = 0; i < m; i++)
        dp[i][0] = 1;
    for (int j = 0; j < n; j++)
        dp[0][j] = 1;
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
    return dp[m-1][n-1];
}
```

CONTINUO...

Capitolo 5

Esercizi

5.1 Esercizi venerdì 13/03/2026

Esercizio 1

Si consideri una lista di interi A (non vuota) di dimensione n contenente tutti gli interi compresi tra 1 e $n+1$ in cui vi è un unico buco (ossia in cui manca un unico elemento). Assumendo che A sia ordinato in modo crescente

1. scrivere lo pseudo-codice di un algoritmo efficiente che restituisce il valore mancante
 - es. se $A=[1, 2, 3, 5, 6, 7]$ l'algoritmo deve restituire 4
2. implementare l'algoritmo proposto in JAVA
3. determinare il costo computazionale dell'algoritmo proposto al punto 1

1.

```
Trova_buco(A,i,j)
    if(A[i] != i e |A| > 1)
        return i
    if(A[j] = j e |A| > 1)
        return j+1
    if(|A|=1)
        if(A[i] != i)
            return 1
        else return 2
    q = punto medio tra i e j
    if q = A[q]
        Trova_buco(q+1,j)
    else
        Trova_buco(i,q)
```

All'inizio mi trovo on $i=1$ e $j=7$, nel primo if controllo se c'è il buco nel primo elemento e lo restituisco. Nel secondo if controllo se c'è il buco nell'ultimo elemento.

Esercizio 2

Consideriamo due liste A e B contenenti rispettivamente n e m interi. Le due liste sono ordinate in senso crescente e ciascuna lista contiene elementi distinti. Tuttavia, lo stesso valore v può comparire sia in A che in B

1. Scrivere lo pseudo-codice di un algoritmo efficiente che sia in grado di stampare tutti i valori di A che **NON** sono presenti in B
 - es. Se $A = [1, 3, 4, 7, 8]$ e $B = [2, 3, 5, 7]$ l'algoritmo deve stampare 1, 4, 8
2. implementare la soluzione proposta in JAVA
3. Determinare il costo computazionale dell'algoritmo presentato al punto 1

1.

$A = [1, 3, 4, 7, 8]$ con $|A| = n$

$B = [2, 3, 5, 7]$ con $|B| = m$

Procedura merge-sort, uso indice i per A e j per B :

$A = [1, 3, 4, 7, 8]$
^
|
 i

$B = [2, 3, 5, 7]$
^
|
 j

Questo è lo pseudocodice:

```
i = 1
j = 1
while (i <= n AND j <= m)
    if(A[i] < B[j])
        stampo(A[i])
        i = i + 1
    else if (A[i] > B[j])
        j = j + 1
    else
        i = i + 1
        j = j + 1
while(i <= n)
    stampo A[i]
    i = i + 1
```

da correggere per quando ci sono i duplicati in A

Costo algoritmo $n + m$

Se non era ordinato comunque aveva un vantaggio prima ordinare le due liste e poi applicare il merge invece di fare la ricerca binaria

Esercizio 3

Consideriamo un array A di numeri reali di dimensione n.

1. Scrivere lo pseudo-codice di un algoritmo che permuti A in modo tale che tutti i valori negativi compaiano prima di tutti quelli positivi. Non è necessario che i valori negati e positivi siano a loro volta ordinati.
2. Implementare l'algoritmo proposto in JAVA
3. determinare il costo computazionale dell'algoritmo proposta al punto 1.

1.

In questo caso usiamo Quick sort con pivot=0.

```
A = [-1, 7, 15, -3, 2, 8, -10]
      ^ ^
      | |
      i j
```

```
A = [-1, 7, 15, -3, 2, 8, -10]
      ^
      |
      j,i
```

```
A = [-1, 7, 15, -3, 2, 8, -10]
      ^           ^
      |           |
      i           j
```

pseudocodice

```
while(j <= |A|)
  if(A[j] < 0)
    swap(A[i+1], A[j])
    i++
  j++
```